

User Manual

for S32K1_S32M24X ICU Driver

Document Number: UM2ICUASRR21-11 Rev0000R3.0.0 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	4
2.3 About This Manual	4
2.4 Acronyms and Definitions	5
2.5 Reference List	5
3 Driver	7
3.1 Requirements	7
3.2 Driver Design Summary	7
3.3 Hardware Resources	8
3.4 Deviations from Requirements	8
3.5 Driver Limitations	9
3.6 Driver usage and configuration tips	9
3.6.1 Icu with DMA feature	9
3.6.2 Dual Clock Feature	10
3.7 Runtime Errors	11
3.8 Symbolic Names Disclaimer	11
3.9 Usage in non AUTOSAR context	11
4 Configuration Parameters	14
4.1 Icu	15
4.1.1 Module Icu	18
4.1.2 Container IcuConfigSet	18
4.1.3 Parameter IcuMaxChannel	19
4.1.4 Container IcuChannel	19
4.1.5 Parameter IcuChannelId	20
4.1.6 Parameter IcuDMAChannelEnable	20
4.1.7 Parameter IcuDefaultStartEdge	21
4.1.8 Parameter IcuMeasurementMode	21
4.1.9 Parameter IcuOverflowNotification	22
4.1.10 Parameter IcuWakeupCapability	22
4.1.11 Reference IcuChannelEcucPartitionRef	23
4.1.12 Reference IcuChannelRef	23
4.1.13 Reference IcuDMAChannelRef	24
4.1.14 Container IcuSignalEdgeDetection	24
4.1.15 Parameter IcuSignalNotification	24
4.1.16 Container IcuSignalMeasurement	25
4.1.17 Parameter IcuSignalMeasurementProperty	25
4.1.18 Container IcuTimestampMeasurement	26
4.1.19 Parameter IcuTimestampMeasurementProperty	26
4.1.20 Parameter IcuTimestampNotification	26
4.1.21 Container IcuWakeup	27
4.1.22 Reference IcuChannelWakeupInfo	27
4.1.23 Container IcuFtm	28
4.1.24 Parameter IcuFtmModule	28
4.1.25 Parameter IcuFtmClockSource	28
4.1.26 Parameter IcuFtmPrescaler	29
4.1.27 Parameter IcuFtmPrescalerAlternate	29
4.1.28 Parameter IcuFtmDebugMode	30
4.1.29 Parameter IcuFtmModValue	30
4.1.30 Container IcuFtmChannels	31

4.1.31	Parameter	IcuFtmChannel	31
4.1.32	Parameter	IcuFtmFilter	31
4.1.33	Container	IcuLpit	32
4.1.34	Parameter	IcuLpitModule	32
4.1.35	Parameter	IcuLpitDebugEnabled	33
4.1.36	Parameter	IcuLpitDozeEnable	33
4.1.37	Container	IcuLpitChannels	33
4.1.38	Parameter	IcuLpitChannel	34
4.1.39	Parameter	IcuLpitTriggerSource	34
4.1.40	Parameter	IcuLpitTriggerSelect	35
4.1.41	Container	IcuLptmr	35
4.1.42	Parameter	IcuLptmrModule	35
4.1.43	Parameter	PrescalerEnable	36
4.1.44	Parameter	IcuLptmrPrescaler	36
4.1.45	Parameter	IcuLptmrChannelClkSrc	37
4.1.46	Parameter	IcuLptmrPinSelect	38
4.1.47	Container	IcuLptmrChannels	38
4.1.48	Parameter	IcuLptmrChannel	39
4.1.49	Container	IcuPort	39
4.1.50	Parameter	IcuPortModule	39
4.1.51	Container	IcuPortChannels	40
4.1.52	Parameter	IcuPortChannel	40
4.1.53	Container	IcuLpCmp	40
4.1.54	Parameter	IcuCmpInstanceNumber	41
4.1.55	Container	IcuCmp	41
4.1.56	Parameter	IcuCmpFunctionalMode	42
4.1.57	Parameter	IcuCmpHysteresisLevel	42
4.1.58	Parameter	IcuCmpOffsetLevel	42
4.1.59	Parameter	IcuCmpEnablePinOutput	43
4.1.60	Parameter	IcuCmpEnableInverter	43
4.1.61	Parameter	IcuCmpEnableHighPowerMode	44
4.1.62	Parameter	IcuCmpFilterSamplePeriod	44
4.1.63	Parameter	IcuCmpFilterSampleCount	44
4.1.64	Parameter	IcuCmpEnableDma	45
4.1.65	Parameter	IcuCmpNegativeInputMux	45
4.1.66	Parameter	IcuCmpPositiveInputMux	46
4.1.67	Parameter	IcuCmpOutputSelect	46
4.1.68	Container	IcuDac	47
4.1.69	Parameter	IcuDacVoltageLevel	47
4.1.70	Parameter	IcuDacVoltageRefSource	48
4.1.71	Parameter	IcuDacPowerState	48
4.1.72	Container	IcuTrigger	48
4.1.73	Parameter	IcuTrgRoundRobinEnChannelMask	49
4.1.74	Parameter	IcuTrgRoundRobinPreSetChannel	49
4.1.75	Parameter	IcuTrgInitDelayValue	50
4.1.76	Parameter	IcuTrgSampleDelay	50
4.1.77	Parameter	IcuTrgFixedChannel	51
4.1.78	Parameter	IcuTrgFixedPort	51
4.1.79	Parameter	IcuTrgEnableRoundRobinInterrupt	52
4.1.80	Parameter	IcuTrgEnableRoundRobin	52
4.1.81	Container	IcuHwInterruptConfigList	52
4.1.82	Parameter	IcuIsrHwId	53
4.1.83	Container	IcuGeneral	53
4.1.84	Parameter	IcuDevErrorDetect	54

4.1.85	Parameter IcuReportWakeupSource	54
4.1.86	Parameter IcuEnableUserModeSupport	54
4.1.87	Parameter IcuMultiPartitionSupport	55
4.1.88	Reference IcuEcucPartitionRef	55
4.1.89	Reference IcuKernelEcucPartitionRef	56
4.1.90	Container IcuAutosarExt	56
4.1.91	Parameter IcuEnableDualClockMode	56
4.1.92	Parameter IcuOverflowNotificationApi	57
4.1.93	Parameter IcuGetInputLevelApi	58
4.1.94	Parameter IcuLptmrStandbyWakeupSupport	58
4.1.95	Parameter IcuGetCaptureRegisterValueApi	58
4.1.96	Container IcuOptionalApis	59
4.1.97	Parameter IcuDeInitApi	59
4.1.98	Parameter IcuDisableWakeupApi	60
4.1.99	Parameter IcuEdgeCountApi	60
4.1.100	Parameter IcuEnableWakeupApi	60
4.1.101	Parameter IcuGetDutyCycleValuesApi	61
4.1.102	Parameter IcuGetInputStateApi	61
4.1.103	Parameter IcuGetTimeElapsedApi	62
4.1.104	Parameter IcuGetVersionInfoApi	62
4.1.105	Parameter IcuSetModeApi	63
4.1.106	Parameter IcuSignalMeasurementApi	63
4.1.107	Parameter IcuTimestampApi	64
4.1.108	Parameter IcuWakeupFunctionalityApi	64
4.1.109	Parameter IcuEdgeDetectApi	64
4.1.110	Container CommonPublishedInformation	65
4.1.111	Parameter ArReleaseMajorVersion	65
4.1.112	Parameter ArReleaseMinorVersion	66
4.1.113	Parameter ArReleaseRevisionVersion	66
4.1.114	Parameter ModuleId	66
4.1.115	Parameter SwMajorVersion	67
4.1.116	Parameter SwMinorVersion	67
4.1.117	Parameter SwPatchVersion	68
4.1.118	Parameter VendorApiInfix	68
4.1.119	Parameter VendorId	69
4.2	LPIT_ICU	69
4.2.1	Module Lpit_Icu	70
4.3	PORT_CI_ICU	70
4.3.1	Module Port_Ci_Icu	71
4.4	FTM_ICU	71
4.4.1	Module Ftm_Icu	72
4.5	LPTMR_ICU	72
4.5.1	Module Lptmr_Icu	73
4.6	CMP	73
4.6.1	Module Cmp	74
5	Module Index	75
5.1	Software Specification	75
6	Module Documentation	76
6.1	LPIT IPL	76
6.1.1	Detailed Description	76
6.1.2	Data Structure Documentation	77
6.1.2.1	struct Lpit_Icu_Ip_ChannelConfigType	77

6.1.2.2 struct Lpit_Icu_Ip_ConfigType	77
6.1.2.3 struct Lpit_Icu_Ip_ChannelsStateType	78
6.1.3 Macro Definition Documentation	78
6.1.3.1 ICU_STOP_SEC_CODE	78
6.1.4 Types Reference	78
6.1.4.1 Lpit_Icu_Ip_NotifyType	78
6.1.4.2 Lpit_Icu_Ip_CallbackType	79
6.1.4.3 Lpit_Icu_Ip_LogicChState	79
6.1.5 Enum Reference	79
6.1.5.1 Lpit_Icu_Ip_StatusType	79
6.1.5.2 Lpit_Icu_Ip_TimestampBufferType	79
6.1.5.3 Lpit_Icu_Ip_MeasurementMode	80
6.1.6 Function Reference	80
6.1.6.1 Lpit_Icu_Ip_Init()	80
6.1.6.2 Lpit_Icu_Ip_EnableInterrupt()	80
6.1.6.3 Lpit_Icu_Ip_DisableInterrupt()	81
6.1.6.4 Lpit_Icu_Ip_EnableNotification()	81
6.1.6.5 Lpit_Icu_Ip_DisableNotification()	81
6.1.6.6 Lpit_Icu_Ip_GetTimestampIndex()	82
6.2 FTM IPL	83
6.2.1 Detailed Description	83
6.2.2 Data Structure Documentation	84
6.2.2.1 struct Ftm_Icu_Ip_DutyCycleType	84
6.2.2.2 struct Ftm_Icu_Ip_ChannelConfigType	84
6.2.2.3 struct Ftm_Icu_Ip_InstanceConfigType	85
6.2.2.4 struct Ftm_Icu_Ip_ConfigType	85
6.2.2.5 struct Ftm_Icu_Ip_ChStateType	86
6.2.2.6 struct Ftm_Icu_Ip_InstStateType	86
6.2.3 Macro Definition Documentation	87
6.2.3.1 CHAN0_IDX	87
6.2.3.2 CHAN1_IDX	87
6.2.3.3 CHAN2_IDX	87
6.2.3.4 CHAN3_IDX	87
6.2.3.5 CHAN4_IDX	87
6.2.3.6 FTM_MAX_VAL_COUNTER	88
6.2.3.7 FTM_COMBINE_COMBINEx_SHIFT	88
6.2.4 Types Reference	88
6.2.4.1 Ftm_Icu_Ip_NotifyType	88
6.2.4.2 Ftm_Icu_Ip_CallbackType	88
6.2.4.3 Ftm_Icu_Ip_LogicChState	88
6.2.5 Enum Reference	88
6.2.5.1 Ftm_Icu_Ip_ClockSourceType	88
6.2.5.2 Ftm_Icu_Ip_DebugModeType	89
6.2.5.3 Ftm_Icu_Ip_EdgeType	89
6.2.5.4 Ftm_Icu_Ip_ModeType	89
6.2.5.5 Ftm_Icu_Ip_SubModeType	90
6.2.5.6 Ftm_Icu_Ip_MeasType	90
6.2.5.7 Ftm_Icu_Ip_LevelType	90
6.2.5.8 Ftm_Icu_Ip_ClockModeType	91
6.2.5.9 Ftm_Icu_Ip_StatusType	91
6.2.5.10 Ftm_Icu_Ip_TimestampBufferType	91
6.3 LPTMR IPL	92
6.3.1 Detailed Description	92
6.3.2 Data Structure Documentation	93

6.3.2.1 struct Lptmr_Icu_Ip_ChannelConfigType	93
6.3.2.2 struct Lptmr_Icu_Ip_ConfigType	93
6.3.3 Macro Definition Documentation	94
6.3.3.1 ICU_STOP_SEC_CODE	94
6.3.3.2 LPTMR_ICU_IP_TM_MODE	94
6.3.4 Types Reference	94
6.3.4.1 Lptmr_Icu_Ip_NotifyType	94
6.3.4.2 Lptmr_Icu_Ip_CallbackType	94
6.3.5 Enum Reference	94
6.3.5.1 Lptmr_Icu_PrescalerType	95
6.3.5.2 Lptmr_Icu_MeasurementModeType	96
6.3.5.3 Lptmr_Icu_Ip_PinSelectType	96
6.3.5.4 Lptmr_Icu_Ip_ClockSourceType	97
6.3.5.5 Lptmr_Icu_Ip_EdgeType	97
6.3.5.6 Lptmr_Icu_Ip_StatusType	97
6.3.6 Function Reference	97
6.3.6.1 Lptmr_Icu_Ip_Init()	97
6.3.6.2 Lptmr_Icu_Ip_SetActivationCondition()	98
6.3.6.3 Lptmr_Icu_Ip_EnableNotification()	98
6.3.6.4 Lptmr_Icu_Ip_DisableNotification()	98
6.4 PORT_CI IPL	99
6.4.1 Detailed Description	99
6.4.2 Data Structure Documentation	100
6.4.2.1 struct Port_Ci_Icu_Ip_ConfigType	100
6.4.2.2 struct Port_Ci_Icu_Ip_State	100
6.4.3 Types Reference	100
6.4.3.1 Port_Ci_Icu_Ip_NotifyType	100
6.4.3.2 Port_Ci_Icu_Ip_CallbackType	100
6.4.4 Enum Reference	100
6.4.4.1 Port_Ci_Icu_Ip_EdgeType	101
6.4.4.2 Port_Ci_Icu_Ip_StatusType	101
6.4.5 Function Reference	101
6.4.5.1 Port_Ci_Icu_Ip_EnableNotification()	101
6.4.5.2 Port_Ci_Icu_Ip_DisableNotification()	101
6.5 CMP IPL	102
6.5.1 Detailed Description	102
6.5.2 Data Structure Documentation	102
6.5.2.1 struct Cmp_Ip_StateType	102
6.5.3 Enum Reference	102
6.5.3.1 Cmp_Ip_StateEType	102
6.6 Icu Driver	103
6.6.1 Detailed Description	103
6.6.2 Data Structure Documentation	106
6.6.2.1 struct Icu_ChannelConfigType	106
6.6.2.2 struct Icu_ConfigType	108
6.6.2.3 struct Icu_DutyCycleType	109
6.6.3 Macro Definition Documentation	110
6.6.3.1 ICU_STOPTIMESTAMP_ID	110
6.6.3.2 ICU_E_NOT_STARTED	110
6.6.4 Types Reference	110
6.6.4.1 Icu_ChannelStateType	110
6.6.4.2 Icu_ChannelType	110
6.6.4.3 Icu_ValueType	111
6.6.4.4 Icu_IndexType	111

6.6.4.5	Icu_EdgeNumberType	111
6.6.4.6	Icu_WakeupValueType	111
6.6.4.7	Icu_MeasurementSubModeType	111
6.6.4.8	Icu_NotifyType	111
6.6.5	Enum Reference	111
6.6.5.1	Icu_ModeType	112
6.6.5.2	Icu_InputStateType	112
6.6.5.3	Icu_MeasurementModeType	112
6.6.5.4	Icu_SignalMeasurementPropertyType	112
6.6.5.5	Icu_TimestampBufferType	113
6.6.5.6	Icu_ActivationType	113
6.6.5.7	Icu_LevelType	113
6.6.5.8	Icu_SelectPrescalerType	114
6.6.6	Function Reference	114
6.6.6.1	Icu_Init()	114
6.6.6.2	Icu_DeInit()	114
6.6.6.3	Icu_SetMode()	115
6.6.6.4	Icu_DisableWakeup()	115
6.6.6.5	Icu_EnableWakeup()	116
6.6.6.6	Icu_CheckWakeup()	116
6.6.6.7	Icu_SetActivationCondition()	116
6.6.6.8	Icu_DisableNotification()	117
6.6.6.9	Icu_EnableNotification()	117
6.6.6.10	Icu_GetInputState()	118
6.6.6.11	Icu_StartTimestamp()	118
6.6.6.12	Icu_StopTimestamp()	119
6.6.6.13	Icu_GetTimestampIndex()	119
6.6.6.14	Icu_ResetEdgeCount()	120
6.6.6.15	Icu_EnableEdgeCount()	120
6.6.6.16	Icu_DisableEdgeCount()	120
6.6.6.17	Icu_GetEdgeNumbers()	121
6.6.6.18	Icu_EnableEdgeDetection()	121
6.6.6.19	Icu_DisableEdgeDetection()	122
6.6.6.20	Icu_GetTimeElapsed()	122
6.6.6.21	Icu_GetDutyCycleValues()	123
6.6.6.22	Icu_StartSignalMeasurement()	123
6.6.6.23	Icu_StopSignalMeasurement()	123
6.6.6.24	Icu_GetVersionInfo()	124
6.6.6.25	Icu_SetClockMode()	124
6.6.6.26	Icu_GetInputLevel()	125
6.6.6.27	Icu_GetCaptureRegisterValue()	125
6.6.7	Variable Documentation	125
6.6.7.1	Icu_pCfgPtr	125
6.6.7.2	Icu_CurrentMode	126
6.6.7.3	Icu_aChannelState	126

Chapter 1

Revision History

Revision	Date	Author	Description
1.0	07.03.2025	NXP RTD Team	S32K1_S32M24X Real-Time Drivers AUTOSAR R21-11 Version 3.0.0

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This User Manual describes NXP Semiconductor AUTOSAR ICU for S32K1_S32M24X. AUTOSAR ICU driver configuration parameters and deviations from the specification are described in ICU Driver chapter of this document. AUTOSAR ICU driver requirements and APIs are described in the AUTOSAR ICU driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32k116_qfn32
- s32k116_lqfp48
- s32k118_lqfp48
- s32k118_lqfp64
- s32k142_lqfp48
- s32k142_lqfp64
- s32k142_lqfp100
- s32k142w_lqfp48
- s32k142w_lqfp64
- s32k144_lqfp48
- s32k144_lqfp64 / MWCT1014S_lqfp64
- s32k144_lqfp100 / MWCT1014S_lqfp100
- s32k144_mapbga100
- s32k144w_lqfp48
- s32k144w_lqfp64

- s32k146_lqfp64
- s32k146_lqfp100 / MWCT1015S_lqfp100
- s32k146_mapbga100 / MWCT1015S_mapbga100
- s32k146_lqfp144
- s32k148_lqfp100
- s32k148_mapbga100 / MWCT1016S_mapbga100
- s32k148_lqfp144
- s32k148_lqfp176
- s32m241_lqfp64
- s32m242_lqfp64
- s32m243_lqfp64
- s32m244_lqfp64

All of the above microcontroller devices are collectively named as S32K1_S32M24X. Note: MWCT part numbers contain NXP confidential IP for Qi Wireless Power

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
ASM	Assembler
BSMI	Basic Software Make file Interface
CAN	Controller Area Network
C/CPP	C and C++ Source Code
LPCMP	Low Power Comparator
CS	Chip Select
CTU	Cross Trigger Unit
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DMA	Direct Memory Access
ECU	Electronic Control Unit
EMIOS	Enhanced Modular IO Subsystem
FIFO	First In First Out
FTM	Flextimer Module
ICU	Input Capture Unit
ISR	Interrupt Service Routine
LSB	Least Significant Bit
MCU	Micro Controller Unit
MIDE	Multi Integrated Development Environment
MSB	Most Significant Bit
N/A	Not Applicable
OS	Operating System
PB Variant	Post Build Variant
PC Variant	Pre Compile Variant
RAM	Random Access Memory
ROM	Read-only Memory
SIUL2	System Integration Unit Lite2
SWS	Software Specification
VLE	Variable Length Encoding
WKPU	Wakeup Unit
XML	Extensible Markup Language

2.5 Reference List

#	Title	Version
1	Specification of Driver	AUTOSAR Release R21-11
2	S32K1xx Reference Manual	Rev.14.1, 01/2024
3	S32M24x Reference Manual	Rev. 5, 12/2024
4	S32K1xx Data Sheet	Rev. 14, 08/2021
5	S32M2xx Data Sheet	Rev. 7, 12/2024

#	Title	Version
6	S32K116 Errata	Mask Set Errata for Mask 0N96V Rev. 17/JAN/2024, 1/2024
7	S32K118 Errata	Mask Set Errata for Mask 0N97V Rev. 15/JAN/2024, 1/2024
8	S32K142 Errata	Mask Set Errata for Mask 0N33V Rev. 15/JAN/2024, 1/2024
9	S32K144 Errata	Mask Set Errata for Mask 0N57U Rev. 15/JAN/2024, 1/2024
10	S32K144W Errata	Mask Set Errata for Mask 0P64A Rev. 03/JUL/2024, 7/2024
11	S32K146 Errata	Mask Set Errata for Mask 0N73V Rev. 15/JAN/2024, 1/2024
12	S32K148 Errata	Mask Set Errata for Mask 0N20V Rev. 15/JAN/2024, 1/2024
13	S32M242 Errata	Mask Set Errata for Mask N33V+P69K Rev.1, 8/2024
14	S32M244 Errata	Mask Set Errata for Mask P64A+P69K Rev.1, 8/2024

Chapter 3

Driver

- [Requirements](#)
- [Driver Design Summary](#)
- [Hardware Resources](#)
- [Deviations from Requirements](#)
- [Driver Limitations](#)
- [Driver usage and configuration tips](#)
- [Runtime Errors](#)
- [Symbolic Names Disclaimer](#)
- [Usage in non AUTOSAR context](#)

3.1 Requirements

Requirements for this driver are detailed in the AUTOSAR R21-11 ICU Driver Software Specification document (See [Table Reference_list](#)).

Requirements for this driver are detailed in the Autosar Driver Software Specification document (See [Table Reference List](#)).

It has vendor-specific requirements and implementation.

3.2 Driver Design Summary

The ICU Driver controls the input capture of the microcontroller. It provides the following features:

- High time / Low time measurement
- Duty Cycle measurement
- Period time measurement
- Edge detection and notification
- Edge counting (with or without hardware gating)
- Edge time stamping
- Wake-up interrupts

For signal edge detection, the edge detector of a capture compare unit or the interrupt controller for external events are used.

For signal measuring a capture timer and at least one capture register are needed. Also, only even channels ($2*n$) can be used for signal measurements. This is because the channel after it ($2*n+1$) is used internally by the ICU Driver

The FTM module supports period time measurement, edge detection and notification, edge counting and edge time stamping.

The PORT_CI module supports edge detection and notification.

The LPTMR module supports edge detection and notification, edge counting.

The LPIT module can support edge detection and notification and edge time stamping.

The ICU driver provides an optional API and configuration parameters for changing the base clock of the controlled hardware. A dual clock functionality is offered by switching between two configured values of the clock prescaler.

For each user configured channel, a symbolic name is generated by the Tresos Studio configuration tool. The name shall be consequently used in upper applications.

By default all channels offer interrupt handlers. For each channel not configured by the user in Tresos Studio configuration tool, the code for interrupt handling is removed based on a series of `#ifdefs`.

The RTD driver assures reentrancy (single core execution) for the APIs based on the following assumptions:

- the "called-again" API is for a different resource (hardware/logic channel);
- common variables/registers accessed with "rmw" are guarded by Exclusive Areas which need to be correctly implemented in RTE on user side;

3.3 Hardware Resources

The hardware resources configured by the Icu driver are next: **CMP**, **LPIT**, **FTM**, **PORT_CI** and **LPTMR**. The CMP, LPIT, FTM, PORT_CI and LPTMR input signal to microcontroller pin mapping can be done by using "IO_Signal_Description_and_Input_multiplexing_tables.xls" from the Reference manual.

3.4 Deviations from Requirements

The driver deviates from the AUTOSAR ICU Driver software specification in some places. The table below identifies the AUTOSAR requirements that are not implemented or out of scope for the ICU Driver.

Term	Definition
N/S	Out of scope
N/I	Not implemented
N/F	Not fully implemented

Below table identifies the AUTOSAR requirements that are not fully implemented, implemented differently or out of scope for the ICU driver.

Requirement	Status	Description	Notes
SWS_Icu_00150	N/S	The Icu module shall not check the integrity if several calls for the same ICU channel are used during runtime in different tasks or ISRs.	Rejection Reason: The requirement is violating safety because: The ICU149 is a safety integrity assumption for external environment, which shall be implemented for FTE; For GTE and NTE ICU149 has a role to increase availability because the check will be supported by ICU driver; see also 00149
SWS_Icu_00380	N/S	These requirements are not applicable to this specification.	Not a requirement
SWS_Icu_91002	N/S	This function disables the notification of a channel. Tags: atp.↵ Status=draft	Description specified as draft is not clear. Should be re-assessed on next ASR version
SWS_Icu_91003	N/S	This function enables the notification on the given channel.↵ Tags: atp.Status=draft	Description specified as draft is not clear. Should be re-assessed on next ASR version
SWS_Icu_CONSTR_00003	N/S	If IcuEcucPartitionRef references one or more ECUC partitions, IcuKernelEcuc↵ PartitionRef shall have a multiplicity of one and reference one of these ECUC partitions as well.	Type IV Autosar multicore not implemented for current module (AAI-445), therefore Icu↵ KernelEcucPartitionRef is not supported.

3.5 Driver Limitations

- Signal measurement with BOTH_EDGE trigger is experimentally supported and returns period and high time only – in analysis to see if speed gain in measurement is valuable.
- The buffer of the timestamp measurement mode may overflow when using the DMA feature if the notification interval (number of events) is smaller than the size of the external buffer (number of entries) in Icu_Start↵ Timestamp api.

3.6 Driver usage and configuration tips

In this chapter, the extra features from our drivers that are not described in the AutoSAR standard are detailed

3.6.1 Icu with DMA feature

In order to speed up data transfer, the Direct Memory Access feature can be used. The DMA feature can be used only in Timestamp mode and is done with the help of the Mcl driver to transfer timestamp data from a Icu input match directly in the Timestamp buffer. Thus, this is an example of Peripheral to Memory data transfer and it's very usefull for avoiding the interrupt overhead. In order to have this feature, the user should check the IcuDMAChannelEnable checkbox for DMA in the Icu channel. This will be selectable only for a channel in Timestamp mode. Also, the user has to configure a Mcl channel with a Mcl Dma Transfer Completion User Notification named <IcuChannelName>_MclDmaTransferCompletionNotif for the respective Icu Channel. for example, for Icu Channel 4, the notification should be named IcuChannel_4_MclDmaTransferCompletionNotif. Apart from that, the

user should configure the DMA instance that allows transfer from that ICU modules. The DMA channel mapping from the RM shows the sources mapping for each peripheral in the DMA channel mapping chapter. This can also be observed in the Mcl plugin in the list of DMA sources. Other fields from the DMA's TCD do not require explicit configuration since they are specific to the ICU's FTM module.

3.6.2 Dual Clock Feature

In order to allow dynamic change of the driver working frequency, the ICU driver has the Dual Clock Feature. The `IcuEnableDualClockMode` from `IcuAutosarExt` should be enabled in order to have this feature active. Afterwards, the `Prescaler_Alternate` parameter allows setting a different prescaler for each module. These parameters will be changed when calling the function call `Icu_SetClockMode`.

`Icu_SetClockMode` may be called only after `Icu_Init` is called and when `IcuEnableDualClockMode` is checked. Our suggested usage of this API is to call it when the driver is in a lower power state but still in active use.

Duty cycle measurement using BOTH EDGES measurement limitation explained.

When this type of measurement is used the limitation consist in considering if the measured `pulseWidth` is always HIGH TIME.

For example, in the next figure it can be seen that for the same signal, `pulseWidth` will have the same value, even for the second one measurement where the active time can be considered the LOW TIME as well.

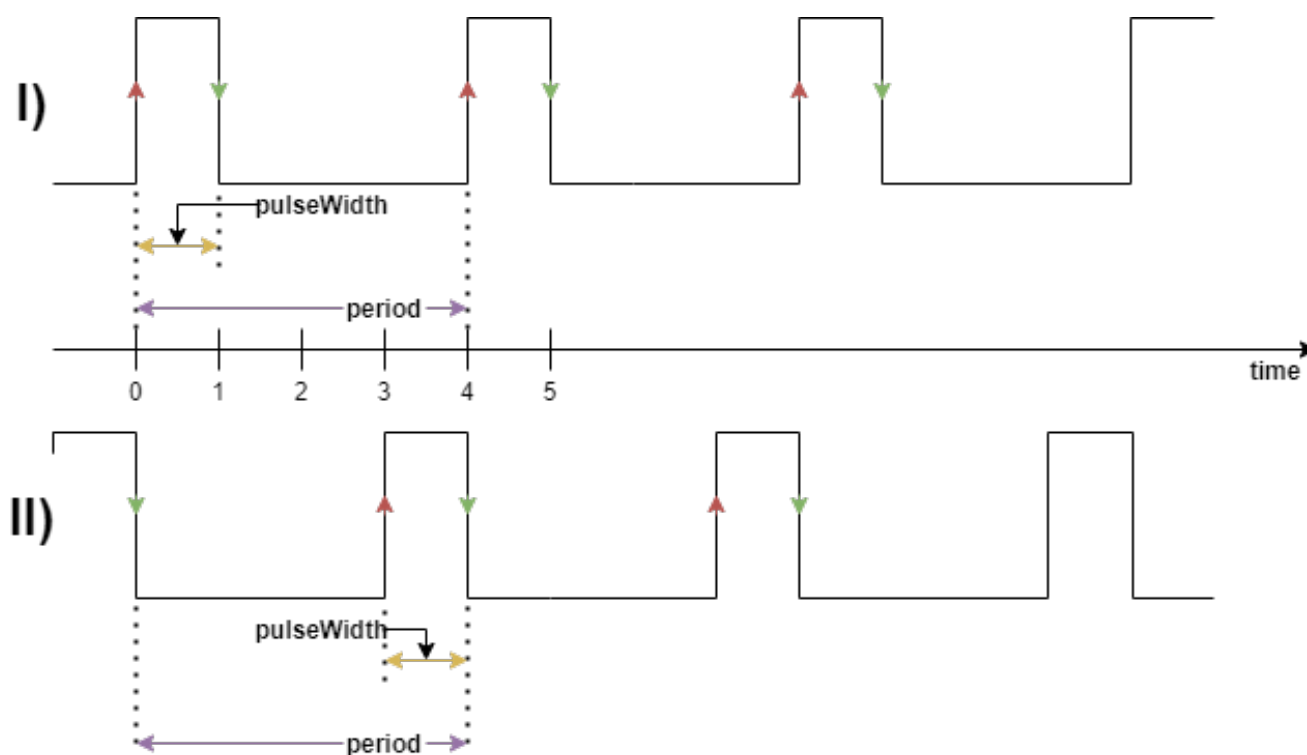
In the I) and the II) case the result will be the same `dutyCycle = 25%`, even the signal will be consider with LOW TIME as the active time.

Pros for this measurement:

\t -> the measurement will be done with no wait time

Cons for this measurement:

\t -> the result will consider the HIGH TIME as the active time for all the measurements with BOTH EDGES measurement, but there is an workaround if it is known that the active time is the LOW TIME `dutyCycle = 100 - currentDutyCycle`(e.g in our case if we consider the LOW TIME as the active time `dutyCycle=100-25=75%`)



Possible measurements for DUTY CYCLE when BOTH EDGES measurement is used

Figure 3.1 Possible measurements for DUTY CYCLE when BOTH EDGES measurement is used

3.7 Runtime Errors

The driver does not generate any DEM runtime errors.

Type of error	Related error code	Error value
API service Icu_StopTimestamp called on a channel which was not started or already stopped	ICU_E_NOT_STARTED	0x15

3.8 Symbolic Names Disclaimer

All containers having symbolicNameValue set to TRUE in the AUTOSAR schema will generate defines like: `#define <Mip>Conf_<Container_ShortName>_<Container_ID>`

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

3.9 Usage in non AUTOSAR context

For the non-ASR environments, both High-Level and IP layer interfaces are exposed and can be used by software units above. These may consist of either higher level NXP SW deliverables, like stacks and middleware, or the application code itself.

The goal of the product is to be versatile, easy to use and integrate with other SW environments, offering both standardized and optimized low-level interfaces.

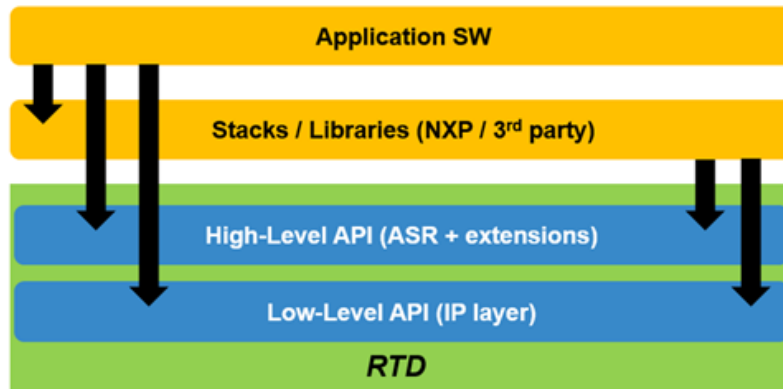


Figure 3.2 RTD Usage in non-ASR Context

In the non-AUTOSAR environment, there are no specific ECU states defined, but an external assumption for the application is that the correct sequence of initialization takes place prior to calling any runtime APIs.

The constraints considered for defining the APIs in the non-AUTOSAR environment are:

- Hardware resources used by the drivers must be properly initialized prior to runtime usage by calling the initialization function with configuration parameters generated by the configuration tools
- All dependencies must be initialized prior to the usage of a driver that depends on them; this translates to a general initialization sequence applicable for all applications, as follows:
 1. Initialization of the device clock tree so other periphery can be initialized
 2. Configuration of the pinout so peripherals with external connections can communicate with the outside world
 3. Configuration of the common resources (e.g. DMA channels, memory regions, access rights, etc) prior to the usage of any runtime functionality that has to benefit from this overall SoC configuration
 4. Initialization of specific drivers
 5. Runtime functionality
 6. Deinitialization (inverse initialization order)

High-level and IP layer functionalities cannot be used simultaneously over the same hardware resource. The granularity of the resource must be defined specifically for each driver, but as a general rule the default granularity considered is the peripheral instance (e.g. Gpt high level APIs cannot be called interchangeably with Pit_Ip low level APIs over the same PIT hardware instance).

The diagram below shows a generic, high-level sequence of calls to RTD interfaces in the non-AUTOSAR scenario:

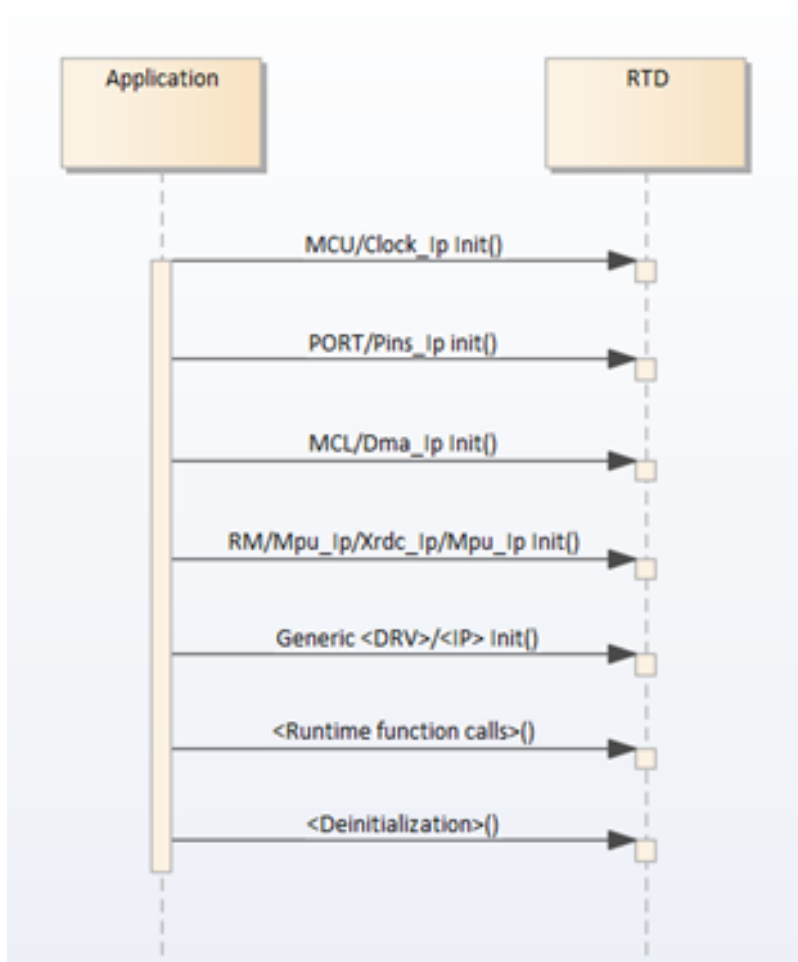


Figure 3.3 Non-AUTOSAR Sequence diagram

Chapter 4

Configuration Parameters

RTD provides configuration schema for the high level driver in EB Tresos proprietary format (**.xdm**), S32CT proprietary format (**.component**), but also in a standardized AUTOSAR format (**.epd**). For the low level driver configuration only the S32CT proprietary format (**.component**) is supported. The configuration data of the project can be saved in EB Tresos proprietary format (**.xdm**) and AUTOSAR standardized format (**.epc**) from EB Tresos and in S32CT proprietary format (**.mex**) and AUTOSAR standardized format (**.epc**) from S32 Design Studio. RTD provides templates for parsing the configuration data and generating configuration structures from it in XPath format, to be used with the EB Tresos generator and in JS format, to be used with the S32CT generator. Since the generation process is not standardized by AUTOSAR, only the two mentioned tools can be used with the provided templates. Using the same input configuration data, the same configuration structures will be obtained using any of the generators.

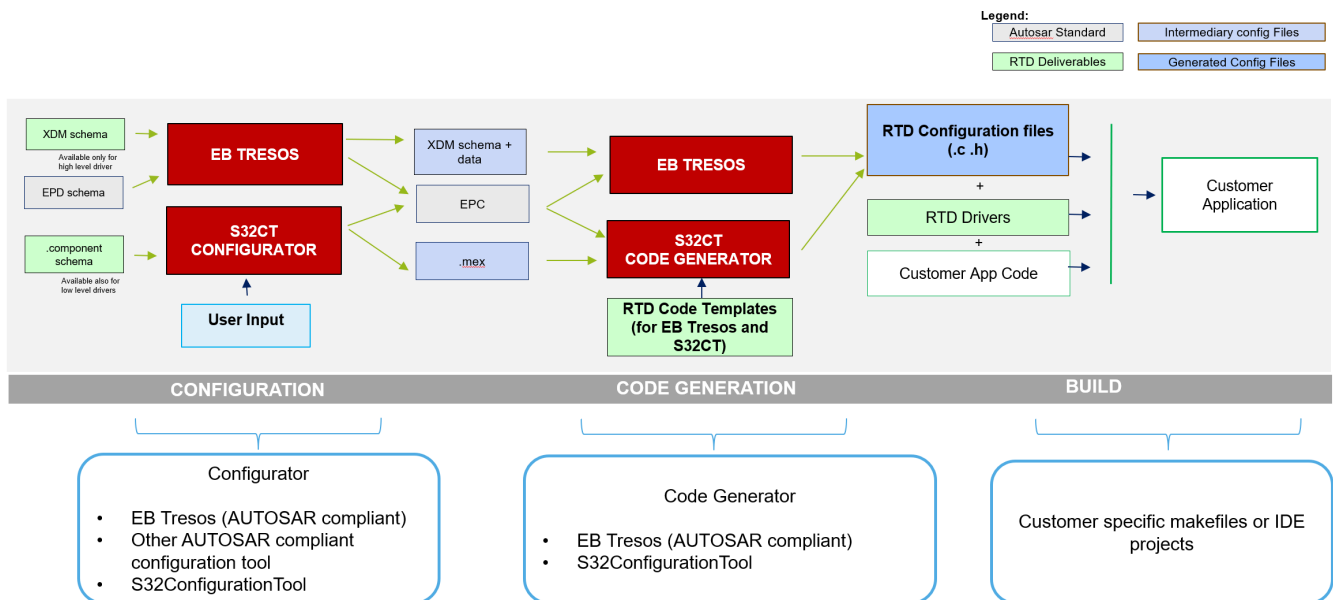


Figure 4.1 Configuration generation flow

- [Icu](#)
- [LPIT_ICU](#)
- [PORT_CI_ICU](#)
- [FTM_ICU](#)

- [LPTMR_ICU](#)
- [CMP](#)

4.1 Icu

- Module [Icu](#)
 - Container [IcuConfigSet](#)
 - * Parameter [IcuMaxChannel](#)
 - * Container [IcuChannel](#)
 - Parameter [IcuChannelId](#)
 - Parameter [IcuDMAChannelEnable](#)
 - Parameter [IcuDefaultStartEdge](#)
 - Parameter [IcuMeasurementMode](#)
 - Parameter [IcuOverflowNotification](#)
 - Parameter [IcuWakeupCapability](#)
 - Reference [IcuChannelEcucPartitionRef](#)
 - Reference [IcuChannelRef](#)
 - Reference [IcuDMAChannelRef](#)
 - Container [IcuSignalEdgeDetection](#)
 - Parameter [IcuSignalNotification](#)
 - Container [IcuSignalMeasurement](#)
 - Parameter [IcuSignalMeasurementProperty](#)
 - Container [IcuTimestampMeasurement](#)
 - Parameter [IcuTimestampMeasurementProperty](#)
 - Parameter [IcuTimestampNotification](#)
 - Container [IcuWakeup](#)
 - Reference [IcuChannelWakeupInfo](#)
 - * Container [IcuFtm](#)
 - Parameter [IcuFtmModule](#)
 - Parameter [IcuFtmClockSource](#)
 - Parameter [IcuFtmPrescaler](#)
 - Parameter [IcuFtmPrescalerAlternate](#)
 - Parameter [IcuFtmDebugMode](#)
 - Parameter [IcuFtmModValue](#)
 - Container [IcuFtmChannels](#)
 - Parameter [IcuFtmChannel](#)
 - Parameter [IcuFtmFilter](#)

- * Container [IcuLpit](#)
 - Parameter [IcuLpitModule](#)
 - Parameter [IcuLpitDebugEnable](#)
 - Parameter [IcuLpitDozeEnable](#)
 - Container [IcuLpitChannels](#)
 - Parameter [IcuLpitChannel](#)
 - Parameter [IcuLpitTriggerSource](#)
 - Parameter [IcuLpitTriggerSelect](#)
- * Container [IcuLptmr](#)
 - Parameter [IcuLptmrModule](#)
 - Parameter [PrescalerEnable](#)
 - Parameter [IcuLptmrPrescaler](#)
 - Parameter [IcuLptmrChannelClkSrc](#)
 - Parameter [IcuLptmrPinSelect](#)
 - Container [IcuLptmrChannels](#)
 - Parameter [IcuLptmrChannel](#)
- * Container [IcuPort](#)
 - Parameter [IcuPortModule](#)
 - Container [IcuPortChannels](#)
 - Parameter [IcuPortChannel](#)
- * Container [IcuLpCmp](#)
 - Parameter [IcuCmpInstanceNumber](#)
 - Container [IcuCmp](#)
 - Parameter [IcuCmpFunctionalMode](#)
 - Parameter [IcuCmpHysteresisLevel](#)
 - Parameter [IcuCmpOffsetLevel](#)
 - Parameter [IcuCmpEnablePinOutput](#)
 - Parameter [IcuCmpEnableInverter](#)
 - Parameter [IcuCmpEnableHighPowerMode](#)
 - Parameter [IcuCmpFilterSamplePeriod](#)
 - Parameter [IcuCmpFilterSampleCount](#)
 - Parameter [IcuCmpEnableDma](#)
 - Parameter [IcuCmpNegativeInputMux](#)
 - Parameter [IcuCmpPositiveInputMux](#)
 - Parameter [IcuCmpOutputSelect](#)
 - Container [IcuDac](#)

- Parameter [IcuDacVoltageLevel](#)
- Parameter [IcuDacVoltageRefSource](#)
- Parameter [IcuDacPowerState](#)
- Container [IcuTrigger](#)
 - Parameter [IcuTrgRoundRobinEnChannelMask](#)
 - Parameter [IcuTrgRoundRobinPreSetChannel](#)
 - Parameter [IcuTrgInitDelayValue](#)
 - Parameter [IcuTrgSampleDelay](#)
 - Parameter [IcuTrgFixedChannel](#)
 - Parameter [IcuTrgFixedPort](#)
 - Parameter [IcuTrgEnableRoundRobinInterrupt](#)
 - Parameter [IcuTrgEnableRoundRobin](#)
- * Container [IcuHwInterruptConfigList](#)
 - Parameter [IcuIsrHwId](#)
- Container [IcuGeneral](#)
 - * Parameter [IcuDevErrorDetect](#)
 - * Parameter [IcuReportWakeupSource](#)
 - * Parameter [IcuEnableUserModeSupport](#)
 - * Parameter [IcuMultiPartitionSupport](#)
 - * Reference [IcuEcucPartitionRef](#)
 - * Reference [IcuKernelEcucPartitionRef](#)
- Container [IcuAutosarExt](#)
 - * Parameter [IcuEnableDualClockMode](#)
 - * Parameter [IcuOverflowNotificationApi](#)
 - * Parameter [IcuGetInputLevelApi](#)
 - * Parameter [IcuLptmrStandbyWakeupSupport](#)
 - * Parameter [IcuGetCaptureRegisterValueApi](#)
- Container [IcuOptionalApis](#)
 - * Parameter [IcuDeInitApi](#)
 - * Parameter [IcuDisableWakeupApi](#)
 - * Parameter [IcuEdgeCountApi](#)
 - * Parameter [IcuEnableWakeupApi](#)
 - * Parameter [IcuGetDutyCycleValuesApi](#)
 - * Parameter [IcuGetInputStateApi](#)
 - * Parameter [IcuGetTimeElapsedApi](#)
 - * Parameter [IcuGetVersionInfoApi](#)

- * Parameter [IcuSetModeApi](#)
- * Parameter [IcuSignalMeasurementApi](#)
- * Parameter [IcuTimestampApi](#)
- * Parameter [IcuWakeupFunctionalityApi](#)
- * Parameter [IcuEdgeDetectApi](#)
- Container [CommonPublishedInformation](#)
 - * Parameter [ArReleaseMajorVersion](#)
 - * Parameter [ArReleaseMinorVersion](#)
 - * Parameter [ArReleaseRevisionVersion](#)
 - * Parameter [ModuleId](#)
 - * Parameter [SwMajorVersion](#)
 - * Parameter [SwMinorVersion](#)
 - * Parameter [SwPatchVersion](#)
 - * Parameter [VendorApiInfix](#)
 - * Parameter [VendorId](#)

4.1.1 Module Icu

Configuration of the Icu (Input Capture Unit) module

Included containers:

- [IcuConfigSet](#)
- [IcuGeneral](#)
- [IcuAutosarExt](#)
- [IcuOptionalApis](#)
- [CommonPublishedInformation](#)

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD, VARIANT-PRE-COMPILE

4.1.2 Container IcuConfigSet

This container is the base for a multiple configuration set

Included subcontainers:

- [IcuChannel](#)
- [IcuFtm](#)
- [IcuLpit](#)

- [IcuLptmr](#)
- [IcuPort](#)
- [IcuLpCmp](#)
- [IcuHwInterruptConfigList](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.3 Parameter IcuMaxChannel

The value for the *IcuMaxChannel* must match with the number of *IcuChannel* configured

For calculating the correct value use the **CALC** button.

Note: Total number of configured channels should be same across all *IcuConfigSets*.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	3
max	32
min	1

4.1.4 Container IcuChannel

Configuration of an individual ICU channel.

Included subcontainers:

- [IcuSignalEdgeDetection](#)
- [IcuSignalMeasurement](#)
- [IcuTimestampMeasurement](#)
- [IcuWakeup](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.5 Parameter IcuChannelId

Channel Id of the ICU channel.

This value will be assigned to the symbolic name derived of the IcuChannel container short name.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	1
max	31
min	0

4.1.6 Parameter IcuDMAChannelEnable

IcuDMAChannelEnable indicates if the corresponding channel will use DMA for measurement

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.1.7 Parameter IcuDefaultStartEdge

Configures the default-activation-edge which shall be used for this channel

if there was no activation-edge configured by the call of service Icu_SetActivationCondition().

In case the Measurement Mode is "IcuSignalMeasurement" and the properties "DutyCycle" or "Period" are set, the edge configured here is used as Default Period Start Edge.

Implementation Type: Icu_ActivationType

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	ICU_RISING_EDGE
literals	['ICU_BOTH_EDGES', 'ICU_FALLING_EDGE', 'ICU_RISING_EDGE']

4.1.8 Parameter IcuMeasurementMode

Configures the measurement mode of this channel.

User should enable optional parameters with respect to the selected IcuMeasurementMode.

Implementation Type: Icu_MeasurementModeType

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	ICU_MODE_SIGNAL_EDGE_DETECT
literals	['ICU_MODE_EDGE_COUNTER', 'ICU_MODE_SIGNAL_EDGE_DETECT', 'ICU_MODE_SIGNAL_MEASUREMENT', 'ICU_MODE_TIMESTAMP']

4.1.9 Parameter IcuOverflowNotification

Icu Overflow Notification Handler

In order to activate this field you have to:

enable IcuOverflowNotificationApi,

choose one of the modes:

ICU_MODE_EDGE_COUNTER,

ICU_MODE_SIGNAL_MEASUREMENT,

ICU_MODE_TIMESTAMP

to enable overflow detection on the internal counter

Note:

Due to hardware implementattion, the Icu Overflow Notification

is not synchronous with the event for ICU_MODE_SIGNAL_MEASUREMENT

and ICU_MODE_TIMESTAMP modes.

The notification will be triggered when measurement completes (for ICU_MODE_SIGNAL_MEASUREMENT)

or the next timestamp event occurs (for ICU_MODE_TIMESTAMP).

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	NULL_PTR

4.1.10 Parameter IcuWakeupCapability

Information about the wakeup-capability of this channel.

true: Channel is wakeup capable.

false: Channel is not wakeup capable.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.1.11 Reference IcuChannelEcucPartitionRef

Maps a ICU channel to zero or multiple ECUC partitions to limit the access to this channel group.

The ECUC partitions referenced are a subset of the ECUC partitions where the ICU driver is mapped to.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.1.12 Reference IcuChannelRef

Select the ICU hw channel on which the functionality of the current ICU channel will be implemented

Property	Value
type	ECUC-CHOICE-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destinations	['/TS_T40D2M30I0R0/Icu/IcuConfigSet/IcuFtm/IcuFtmChannels', '/TS_T40D2M30I0R0/Icu/IcuConfigSet/IcuLpit/IcuLpitChannels', '/TS_T40D2M30I0R0/Icu/IcuConfigSet/IcuLptmr/IcuLptmrChannels', '/TS_T40D2M30I0R0/Icu/IcuConfigSet/IcuPort/IcuPortChannels', '/TS_T40D2M30I0R0/Icu/IcuConfigSet/IcuLpCmp']

4.1.13 Reference IcuDMAChannelRef

Icu DMA Channel Reference

Reference to the DMA Channel configure for the Request

Property	Value
type	ECUC-CHOICE-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False
destinations	['/TS_T40D2M30I0R0/Mcl/MclConfig/dmaLogicChannel_Type']

4.1.14 Container IcuSignalEdgeDetection

This container contains the configuration (parameters) in case the measurement mode is "IcuSignalEdgeDetection"

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

4.1.15 Parameter IcuSignalNotification

Notification function for signal notification

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	NULL_PTR

4.1.16 Container IcuSignalMeasurement

This container contains the configuration (parameters) in case the measurement mode is "IcuSignalMeasurement"

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.17 Parameter IcuSignalMeasurementProperty

Configures the property that could be measured in case the mode is "IcuSignalMeasurement".

This property can not be changed during runtime.

Followings are measurement mode

ICU_DUTY_CYCLE

ICU_HIGH_TIME

ICU_LOW_TIME

ICU_PERIOD_TIME

Implementation Type: Icu_SignalMeasurementPropertyType

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	ICU_DUTY_CYCLE

Configuration Parameters

Property	Value
literals	['ICU_DUTY_CYCLE', 'ICU_HIGH_TIME', 'ICU_LOW_TIME', 'ICU_PERIOD_TIME']

4.1.18 Container IcuTimestampMeasurement

This container contains the configuration (parameters) in case the measurement mode is "IcuTimestamp"

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.19 Parameter IcuTimestampMeasurementProperty

Configures the handling of the buffer in case the mode is "Timestamp"

Following type of buffer implemented in current implementation.

ICU_CIRCULAR_BUFFER.

ICU_LINEAR_BUFFER.

Implementation Type: Icu_TimestampBufferType

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	ICU_LINEAR_BUFFER
literals	['ICU_CIRCULAR_BUFFER', 'ICU_LINEAR_BUFFER']

4.1.20 Parameter IcuTimestampNotification

Notification function if the number of requested timestamps(Notification interval > 0) are acquired

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	NULL_PTR

4.1.21 Container IcuWakeup

This container contains the configuration (parameters) needed to configure a wakeup capable channel

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.22 Reference IcuChannelWakeupInfo

If the wakeup-capability is true the wakeup source referenced is transmitted to the ECU State Manager (EcuM) .

Implementation Type: reference to EcuM_WakeupSourceType

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
requiresSymbolicNameValue	False

Configuration Parameters

Property	Value
destination	/AUTOSAR/EcuDefs/EcuM/EcuMConfiguration/EcuMCommon← Configuration/EcuMWakeupSource

4.1.23 Container IcuFtm

Configuration of a FTM module available on the platform.

Included subcontainers:

- [IcuFtmChannels](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.1.24 Parameter IcuFtmModule

Select which hardware instance of FTM to use.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	3
min	0

4.1.25 Parameter IcuFtmClockSource

Select origin of clock source used by current instance of FTM.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	SYSTEM_CLOCK
literals	['SYSTEM_CLOCK', 'EXTERNAL_CLOCK', 'FIXED_FREQUENCY_CLOCK']

4.1.26 Parameter IcuFtmPrescaler

Prescaler used by current FTM instance.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	1
max	128
min	0

4.1.27 Parameter IcuFtmPrescalerAlternate

Set an alternante prescaler for FTM instance. With this option the frequency can be changed at runtime.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

Property	Value
default Value	128
max	128
min	0

4.1.28 Parameter IcuFtmDebugMode

Mode 0: FTM counter stopped, CH(n)F bit can be set, FTM channels in functional mode, writes to MOD,CNTIN and C(n)V registers bypass the register buffers.

Mode 1: FTM counter stopped, CH(n)F bit is not set, FTM channels outputs are forced to their safe value , writes to MOD,CNTIN and C(n)V registers bypass the register buffers.

Mode 2: FTM counter stopped, CH(n)F bit is not set, FTM channels outputs are frozen when chip enters in BDM mode, writes to MOD, CNTIN and C(n)V registers bypass the register buffers.

Mode 3: FTM counter in functional mode, CH(n)F bit can be set,FTM channels in functional mode, writes to MOD,CNTIN and C(n)V registers is in fully functional mode.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	MODE_0
literals	['MODE_0', 'MODE_1', 'MODE_2', 'MODE_3']

4.1.29 Parameter IcuFtmModValue

Maxim value of counter for current FTM instance. This parameter will define the maxim pulse period which can be measured.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

Property	Value
defaultValue	0
max	65535
min	0

4.1.30 Container IcuFtmChannels

List of Ftm channels available on the platform.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.1.31 Parameter IcuFtmChannel

Select a hardware channel of the current FTM instance to configure.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	7
min	0

4.1.32 Parameter IcuFtmFilter

Input Capture Filter Control: Selects the filter value for the channel input. The filter is disabled when the value is zero.

Note: This is an Implementation Specific Parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF

Configuration Parameters

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	15
min	0

4.1.33 Container IcuLpit

Configuration of a Lpit module available on the platform.

Included subcontainers:

- [IcuLpitChannels](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	2
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.1.34 Parameter IcuLpitModule

Select the physical Lpit Module.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	0

Property	Value
min	0

4.1.35 Parameter IcuLpitDebugEnabled

Vendor specific: Select to set hardware in debug mode.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	false

4.1.36 Parameter IcuLpitDozeEnable

Enables/Disables DOZE Mode.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	true

4.1.37 Container IcuLpitChannels

List of Lpit Channels on platform.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF

Property	Value
lowerMultiplicity	1
upperMultiplicity	4
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.1.38 Parameter IcuLpitChannel

Selects one of the Lpit channels available on the platform.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	0
max	4
min	0

4.1.39 Parameter IcuLpitTriggerSource

Selects the internal and external trigger sources .

`INTERNAL_TRIGGER`

`EXTERNAL_TRIGGER`

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	INTERNAL_TRIGGER
literals	['INTERNAL_TRIGGER', 'EXTERNAL_TRIGGER']

4.1.40 Parameter IcuLpitTriggerSelect

Select one trigger from the set of internal or external triggers selected by TRG_SRC.

ICU_LPIT_ICU_TRG_CH0

ICU_LPIT_ICU_TRG_CH1

ICU_LPIT_ICU_TRG_CH2

ICU_LPIT_ICU_TRG_CH3

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	LPIT_ICU_TRG_CH0
literals	['LPIT_ICU_TRG_CH0', 'LPIT_ICU_TRG_CH1', 'LPIT_ICU_TRG_CH2', 'LPIT_ICU_TRG_CH3']

4.1.41 Container IcuLptmr

Configuration of a Lptmr module available on the platform.

Included subcontainers:

- [IcuLptmrChannels](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	2
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE

4.1.42 Parameter IcuLptmrModule

Select the physical Lptmr Module.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false

Configuration Parameters

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	0
min	0

4.1.43 Parameter PrescalerEnable

When PBYP is set, the selected prescaler clock in Time Counter mode or selected input source in Pulse Counter mode directly clocks the CNR.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	true

4.1.44 Parameter IcuLptmrPrescaler

Vendor specific: The ICU module specific Glitch filter value.

Configures the Glitch filter in Pulse Counter mode.

- 1 - Glitch filter recognizes change on input pin after 2 rising clock edges
- 2 - Glitch filter recognizes change on input pin after 4 rising clock edges
- 3 - Glitch filter recognizes change on input pin after 8 rising clock edges
- 4 - Glitch filter recognizes change on input pin after 16 rising clock edges
- 5 - Glitch filter recognizes change on input pin after 32 rising clock edges
- 6 - Glitch filter recognizes change on input pin after 64 rising clock edges
- 7 - Glitch filter recognizes change on input pin after 128 rising clock edges
- 8 - Glitch filter recognizes change on input pin after 256 rising clock edges
- 9 - Glitch filter recognizes change on input pin after 512 rising clock edges

10 - Glitch filter recognizes change on input pin after 1024 rising clock edges

11 - Glitch filter recognizes change on input pin after 2048 rising clock edges

12 - Glitch filter recognizes change on input pin after 4096 rising clock edges

13 - Glitch filter recognizes change on input pin after 8192 rising clock edges

14 - Glitch filter recognizes change on input pin after 16384 rising clock edges

15 - Glitch filter recognizes change on input pin after 32768 rising clock edges

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	LPTMR_ICU_GLITCH_FILTER_2
literals	['LPTMR_ICU_GLITCH_FILTER_2', 'LPTMR_ICU_GLITCH_FILTER_4', 'LPTMR_ICU_GLITCH_FILTER_8', 'LPTMR_ICU_GLITCH_FILTER_16', 'LPTMR_ICU_GLITCH_FILTER_32', 'LPTMR_ICU_GLITCH_FILTER_64', 'LPTMR_ICU_GLITCH_FILTER_128', 'LPTMR_ICU_GLITCH_FILTER_256', 'LPTMR_ICU_GLITCH_FILTER_512', 'LPTMR_ICU_GLITCH_FILTER_1024', 'LPTMR_ICU_GLITCH_FILTER_2048', 'LPTMR_ICU_GLITCH_FILTER_4096', 'LPTMR_ICU_GLITCH_FILTER_8192', 'LPTMR_ICU_GLITCH_FILTER_16384', 'LPTMR_ICU_GLITCH_FILTER_32768', 'LPTMR_ICU_GLITCH_FILTER_65536']

4.1.45 Parameter IcuLptmrChannelClkSrc

Vendor specific: The ICU module specific clock input for the timer unit can statically be configured and allows to select different clock sources per module.

Select the clock source for the FlexTimer module for this platform.

LPTMR_ICU_SIRCDIV2_CLK

LPTMR_ICU_LPO1K_CLK

LPTMR_ICU_RTC_CLK

LPTMR_ICU_PCC_LPTMR0

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Configuration Parameters

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	LPTMR_ICU_SIRCDIV2_CLK
literals	['LPTMR_ICU_SIRCDIV2_CLK', 'LPTMR_ICU_LPO1K_CLK', 'LPTMR_ICU_RTC_CLK', 'LPTMR_ICU_PCC_LPTMR0']

4.1.46 Parameter IcuLptmrPinSelect

Configures the input source to be used in Pulse Counter mode.

ALT1

ALT2

ALT3

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	ALT1
literals	['TRGMUX_OUTPUT', 'ALT1', 'ALT2', 'ALT3']

4.1.47 Container IcuLptmrChannels

List of Lptmr Channels on platform.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	2
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE

4.1.48 Parameter IcuLptmrChannel

Selects one of the Lptmr channels available on the platform.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	0
min	0

4.1.49 Container IcuPort

Configuration of a Port module available on the platform.

Included subcontainers:

- [IcuPortChannels](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.1.50 Parameter IcuPortModule

Select the physical Port Module.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true

Configuration Parameters

Property	Value
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	4
min	0

4.1.51 Container IcuPortChannels

List of Port Channels on platform.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.1.52 Parameter IcuPortChannel

Selects one of the Port channels available on the platform.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	17
min	0

4.1.53 Container IcuLpCmp

Configuration of a LPCMP module available on the platform.

Included subcontainers:

- [IcuCmp](#)

- [IcuDac](#)
- [IcuTrigger](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	2
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.1.54 Parameter IcuCmpInstanceNumber

Configure the instance number of IP used. *Note This is an Implementation Specific Parameter.*

Property	Value
type	ECUC-INTEGGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	0
min	0

4.1.55 Container IcuCmp

Configuration of a LPCMP module available on the platform.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.56 Parameter IcuCmpFunctionalMode

Functional mode of LPCMP

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	CMP_IP_FUNCTIONALMODE_DISABLED
literals	['CMP_IP_FUNCTIONALMODE_DISABLED', 'CMP_IP_FUNCTIONALMODE_CONTINUOUS', 'CMP_IP_FUNCTIONALMODE_SAMPLED_NONFILTERED_INT_CLK', 'CMP_IP_FUNCTIONALMODE_SAMPLED_NONFILTERED_EXT_CLK', 'CMP_IP_FUNCTIONALMODE_SAMPLED_FILTERED_INT_CLK', 'CMP_IP_FUNCTIONALMODE_SAMPLED_FILTERED_EXT_CLK', 'CMP_IP_FUNCTIONALMODE_WINDOWED', 'CMP_IP_FUNCTIONALMODE_WINDOWED_RESAMPLED', 'CMP_IP_FUNCTIONALMODE_WINDOWED_FILTERED']

4.1.57 Parameter IcuCmpHysteresisLevel

Internal hysteresis mode of LPCMP - see specific implementation

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	CMP_IP_HYSTERESISLEVEL_0
literals	['CMP_IP_HYSTERESISLEVEL_0', 'CMP_IP_HYSTERESISLEVEL_1', 'CMP_IP_HYSTERESISLEVEL_2', 'CMP_IP_HYSTERESISLEVEL_3']

4.1.58 Parameter IcuCmpOffsetLevel

Comparator offset control - see specific implementation

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	CMP_IP_OFFSETLEVEL_0
literals	['CMP_IP_OFFSETLEVEL_0', 'CMP_IP_OFFSETLEVEL_1']

4.1.59 Parameter IcuCmpEnablePinOutput

EnablePinOutput.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.60 Parameter IcuCmpEnableInverter

EnableInverter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.61 Parameter IcuCmpEnableHighPowerMode

EnableHighPowerMode.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.62 Parameter IcuCmpFilterSamplePeriod

FilterSamplePeriod

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
default Value	0
max	255
min	0

4.1.63 Parameter IcuCmpFilterSampleCount

FilterSampleCount

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	7
min	0

4.1.64 Parameter IcuCmpEnableDma

EnableDma.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.65 Parameter IcuCmpNegativeInputMux

NegativeInputMux

Note: S32M24x derivatives only support the following CMP input connections:

*  CMP_IP_INPUTMUX_IN2*

*  CMP_IP_INPUTMUX_IN4*

*  CMP_IP_INPUTMUX_IN5*

*  CMP_IP_INPUTMUX_DAC*

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF

Configuration Parameters

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	CMP_IP_INPUTMUX_DAC
literals	['CMP_IP_INPUTMUX_IN0', 'CMP_IP_INPUTMUX_IN1', 'CMP_IP_INPUTMUX_IN2', 'CMP_IP_INPUTMUX_IN3', 'CMP_IP_INPUTMUX_IN4', 'CMP_IP_INPUTMUX_IN5', 'CMP_IP_INPUTMUX_IN6', 'CMP_IP_INPUTMUX_IN7', 'CMP_IP_INPUTMUX_DAC']

4.1.66 Parameter IcuCmpPositiveInputMux

PositiveInputMux

Note: S32M24x derivatives only support the following CMP input connections:

  CMP_IP_INPUTMUX_IN2

  CMP_IP_INPUTMUX_IN4

  CMP_IP_INPUTMUX_IN5

  CMP_IP_INPUTMUX_DAC

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	CMP_IP_INPUTMUX_DAC
literals	['CMP_IP_INPUTMUX_IN0', 'CMP_IP_INPUTMUX_IN1', 'CMP_IP_INPUTMUX_IN2', 'CMP_IP_INPUTMUX_IN3', 'CMP_IP_INPUTMUX_IN4', 'CMP_IP_INPUTMUX_IN5', 'CMP_IP_INPUTMUX_IN6', 'CMP_IP_INPUTMUX_IN7', 'CMP_IP_INPUTMUX_DAC']

4.1.67 Parameter IcuCmpOutputSelect

OutputSelect

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	CMP_IP_OUTPUTSELECT_COUT
literals	['CMP_IP_OUTPUTSELECT_COUT', 'CMP_IP_OUTPUTSELECT_COUTA']

4.1.68 Container IcuDac

Configuration of a LPCMP module available on the platform.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.69 Parameter IcuDacVoltageLevel

VoltageLevel

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	255
min	0

4.1.70 Parameter IcuDacVoltageRefSource

VoltageRefSource

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	CMP_IP_VOLTAGEREFSSOURCE_VREF0
literals	['CMP_IP_VOLTAGEREFSSOURCE_VREF0', 'CMP_IP_VOLTAGEREFSSOURCE_VREF1']

4.1.71 Parameter IcuDacPowerState

PowerState

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	CMP_IP_DACPOWERSTATE_DISABLED
literals	['CMP_IP_DACPOWERSTATE_DISABLED', 'CMP_IP_DACPOWERSTATE_ENABLED']

4.1.72 Container IcuTrigger

Configuration of a LPCMP module available on the platform.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.73 Parameter IcuTrgRoundRobinEnChannelMask

IcuTrgRoundRobinEnChannelMask

Note:

 S32M24x derivatives only support 3 input channels: 2, 4, 5

 S32K116 derivatives and S32K14X 48 pin only support 6 input channels: 0 -> 5

 Other derivatives support 8 input channels: 0 -> 7

Example Calculate value:

 Config channels 1, 2, 5: 100110 -> fill in: 38

 Config channels 0, 4: 10001 -> fill in: 17

 You can calculate other values similarly and fill in

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	0
max	255
min	0

4.1.74 Parameter IcuTrgRoundRobinPreSetChannel

RoundRobinPreSetChannel

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Configuration Parameters

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	255
min	0

4.1.75 Parameter IcuTrgInitDelayValue

InitDelayValue

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	63
min	0

4.1.76 Parameter IcuTrgSampleDelay

SampleDelay

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	CMP_IP_SAMPLEDELAY_0_CYCLES

Property	Value
literals	['CMP_IP_SAMPLEDELAY_0_CYCLES', 'CMP_IP_SAMPLEDELAY_1_↵ _CYCLES', 'CMP_IP_SAMPLEDELAY_2_CYCLES', 'CMP_IP_SAMPL↵ EDELAY_3_CYCLES']

4.1.77 Parameter IcuTrgFixedChannel

FixedChannel

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	CMP_IP_FIXEDCHANNEL_0
literals	['CMP_IP_FIXEDCHANNEL_0', 'CMP_IP_FIXEDCHANNEL_1', 'CMP_↵ IP_FIXEDCHANNEL_2', 'CMP_IP_FIXEDCHANNEL_3', 'CMP_IP_FIX↵ EDCHANNEL_4', 'CMP_IP_FIXEDCHANNEL_5', 'CMP_IP_FIXEDCHA↵ NNEL_6', 'CMP_IP_FIXEDCHANNEL_7']

4.1.78 Parameter IcuTrgFixedPort

FixedPort

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	CMP_IP_FIXEDPORT_PLUS
literals	['CMP_IP_FIXEDPORT_PLUS', 'CMP_IP_FIXEDPORT_MINUS']

4.1.79 Parameter IcuTrgEnableRoundRobinInterrupt

EnableRoundRobinInterrupt

Note: If you want to use Round-Robin interrupt, CMP interrupt is not enabled (do not use Icu_EnableEdgeDetection and Cmp_lp_EnableInterrupt function).

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.80 Parameter IcuTrgEnableRoundRobin

EnableRoundRobin.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.81 Container IcuHwInterruptConfigList

List of HW interrupts available for the entire platform.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite

Property	Value
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.1.82 Parameter IcuIsrHwId

Id of the HW interrupt service routine available platform wide and usable by ICU module.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	FTM_0_CH_1
literals	['FTM_0_CH_0', 'FTM_0_CH_1', 'FTM_0_CH_2', 'FTM_0_CH_3', 'FTM_0_CH_4', 'FTM_0_CH_5', 'FTM_0_CH_7', 'FTM_1_CH_0', 'FTM_1_CH_2', 'FTM_1_CH_4', 'FTM_1_CH_6', 'FTM_1_CH_7', 'FTM_2_CH_0', 'FTM_2_CH_1', 'FTM_2_CH_2', 'FTM_2_CH_3', 'FTM_2_CH_5', 'FTM_2_CH_6', 'FTM_2_CH_7', 'FTM_3_CH_6', 'FTM_3_CH_7', 'PORT_0', 'PORT_1', 'PORT_2', 'PORT_3', 'PORT_4', 'LPIT_0_CH_0', 'LPIT_0_CH_1', 'LPIT_0_CH_2', 'LPIT_0_CH_3', 'LPTMR_0_CH_0', 'CMP_0']

4.1.83 Container IcuGeneral

Configuration of general ICU parameters.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.84 Parameter IcuDevErrorDetect

Switches the Development Error Detection and Notification on or off.

true: Enabled.

false: Disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.85 Parameter IcuReportWakeupSource

Switch for enabling Wakeup source reporting.

true: Report Wakeup source.

false: Do not report Wakeup source.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.86 Parameter IcuEnableUserModeSupport

When this parameter is enabled, the Icu module will adapt to run from User Mode, with the following measures:

a) configuring REG_PROT for SIUL2 IP so that the registers under protection can be accessed from user mode by setting UAA bit in REG_PROT_GCR to 1

b) using 'call trusted function' stubs for all internal function calls that access registers requiring supervisor mode.

for more information, please see chapter 5.7 User Mode Support in IM

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.87 Parameter IcuMultiPartitionSupport

When this parameter is enabled, the ICU module will adapt to run with Multi-Partition:

true: Multi-Partition support active.

false: Single Partition support.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.88 Reference IcuEcucPartitionRef

Maps the ICU driver to zero or multiple ECUC partitions to make the driver API available in the according partition.

Depending on the addressed timer resource the interfaces operate as follows.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true

Property	Value
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuC/EcuPartitionCollection/EcuPartition

4.1.89 Reference IcuKernelEcuPartitionRef

Maps the ICU kernel to zero or one ECUC partitions to assign the driver kernel to a certain core.

The ECUC partition referenced is a subset of the ECUC partitions where the ICU driver is mapped to.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuC/EcuPartitionCollection/EcuPartition

4.1.90 Container IcuAutosarExt

Enabling the settings of this section will configure the driver in a mode not compliant with AUTOSAR requirements.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.91 Parameter IcuEnableDualClockMode

Enables prescaler settings at mode transition.

true: Enabled.

false: Disabled.

Note: This feature is not required by Autosar.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.92 Parameter IcuOverflowNotificationApi

Add / removes Overflow Notification functionality.

Enabling IcuOverflowNotificationApi overflow events will not be treated as errors and a Notification Handler can be provided.

If this optional API is not enabled, overflow events will trigger DET Report Error.

Note:

Due to hardware implementattion, the Icu Overflow Notification is not synchronous with the event for ICU_MODE_SIGNAL_MEASUREMENT and ICU_MODE_TIMESTAMP modes. The notification will be triggered when measurement completes (for ICU_MODE_SIGNAL_MEASUREMENT) or the next timestamp event occurs (for ICU_MODE_TIMESTAMP).

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.93 Parameter IcuGetInputLevelApi

Add / removes *Icu_GetInputLevel* API from the code.

This function returns Input pin state.

Note: This feature is not required by Autosar.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.94 Parameter IcuLptmrStandbyWakeupSupport

Icu_Init() will not clear the IRQ flags (LPTMR) if it is already set during init.

Note: This feature is not required by Autosar and might overrule standard.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.95 Parameter IcuGetCaptureRegisterValueApi

Adds / removes service *Icu_GetCaptureRegisterValue* from the code.

This function returns value of Capture register for the measurement channel or timestamp mode channel which is called by the user.

It's enabled when *IcuTimestampApi* or *IcuSignalMeasurementApi* is true.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.96 Container IcuOptionalApis

This container contains all configuration switches for configuring optional API services of the ICU driver.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.97 Parameter IcuDeInitApi

Adds / removes the service Icu_DeInit() from the code.

true: Icu_DeInit() can be used.

false: Icu_DeInit() can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.98 Parameter IcuDisableWakeupApi

Adds / removes the service `Icu_DisableWakeup()` from the code.

true: Icu_DisableWakeup() can be used.

false: Icu_DisableWakeup() can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.99 Parameter IcuEdgeCountApi

Adds / removes all services related to the edge counting

functionality as listed below, from the code: `Icu_ResetEdgeCount()`,

`Icu_EnableEdgeCount()`, `Icu_DisableEdgeCount()`, `Icu_GetEdgeNumbers()`.

true: The services listed above can be used.

false: The services listed above can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.100 Parameter IcuEnableWakeupApi

Adds / removes the service `Icu_EnableWakeup()` from the code.

true: Icu_EnableWakeup() can be used.

false: Icu_EnableWakeup() can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.101 Parameter IcuGetDutyCycleValuesApi

Adds / removes the service `Icu_GetDutyCycleValues()` from the code.

true: Icu_GetDutyCycleValues() can be used.

false: Icu_GetDutyCycleValues() can not be used.

Note: If IcuSignalMeasurementApi == OFF this switch is shall also be set to OFF.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.102 Parameter IcuGetInputStateApi

Adds / removes the service `Icu_GetInputState()` from the code.

true: Icu_GetInputState() can be used.

false: Icu_GetInputState() can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.103 Parameter IcuGetTimeElapsedApi

Adds / removes the service `Icu_GetTimeElapsed()` from the code.

true: Icu_GetTimeElapsed() can be used.

false: Icu_GetTimeElapsed() can not be used.

Note: If IcuSignalMeasurementApi == OFF this switch is shall also be set to OFF.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.104 Parameter IcuGetVersionInfoApi

Adds / removes the service `Icu_GetVersionInfo()` from the code.

true: Icu_GetVersionInfo() can be used.

false: Icu_GetVersionInfo() can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false

Property	Value
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.105 Parameter IcuSetModeApi

Adds / removes the service `Icu_SetMode()` from the code.

true: `Icu_SetMode()` can be used.

false: `Icu_SetMode()` can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.106 Parameter IcuSignalMeasurementApi

Adds / removes the services `Icu_StartSignalMeasurement()` and `Icu_StopSignalMeasurement()` from the code.

true: `Icu_StartSignalMeasurement()` and `Icu_StopSignalMeasurement()` can be used.

false: `Icu_StartSignalMeasurement()` and `Icu_StopSignalMeasurement()` can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.107 Parameter IcuTimestampApi

Adds / removes all services related to the timestamping functionality as listed below from the code:

`Icu_StartTimestamp()`, `Icu_StopTimestamp()`, `Icu_GetTimestampIndex()`.

true: The services listed above can be used.

false: The services listed above can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.108 Parameter IcuWakeupFunctionalityApi

Adds / removes the service `Icu_CheckWakeup()` from the code.

true: `Icu_CheckWakeup()` can be used.

false: `Icu_CheckWakeup()` can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.109 Parameter IcuEdgeDetectApi

Adds / removes the services `Icu_EnableEdgeDetection()` and `Icu_DisableEdgeDetection()` from the code.

true: `Icu_EnableEdgeDetection()` and `Icu_DisableEdgeDetection()` can be used.

false: `Icu_EnableEdgeDetection()` and `Icu_DisableEdgeDetection()` can not be used.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.110 Container CommonPublishedInformation

Common container, aggregated by all modules. It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.111 Parameter ArReleaseMajorVersion

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
default Value	4
max	4
min	4

4.1.112 Parameter ArReleaseMinorVersion

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
defaultValue	7
max	7
min	7

4.1.113 Parameter ArReleaseRevisionVersion

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.114 Parameter ModuleId

Module ID of this module from Module List.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
defaultValue	122
max	122
min	122

4.1.115 Parameter SwMajorVersion

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
defaultValue	3
max	3
min	3

4.1.116 Parameter SwMinorVersion

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
	VARIANT-POST-BUILD: PUBLISHED-INFORMATION

Configuration Parameters

Property	Value
defaultValue	0
max	0
min	0

4.1.117 Parameter SwPatchVersion

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION VARIANT-POST-BUILD: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.118 Parameter VendorApiInfix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the implementation specific name is generated as follows: <ModuleName>_<VendorId>_<VendorApiInfix>.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name Can_Write defined in the SWS will translate to Can_123_v11r456Write.

This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION VARIANT-POST-BUILD: PUBLISHED-INFORMATION
postBuildVariantValue	false

Property	Value
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
defaultValue	

4.1.119 Parameter VendorId

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
defaultValue	43
max	43
min	43

4.2 LPIT_ICU

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module [Lpit_Icu](#)
 - Container [IcuConfigSet](#)
 - * Parameter [IcuMaxChannel](#)
 - * Container [IcuChannel](#)
 - Parameter [IcuChannelId](#)
 - Parameter [IcuDefaultStartEdge](#)
 - Parameter [IcuMeasurementMode](#)
 - Parameter [IcuOverflowNotification](#)
 - Reference [IcuChannelRef](#)
 - Container [IcuSignalEdgeDetection](#)
 - Container [IcuSignalMeasurement](#)
 - Container [IcuTimestampMeasurement](#)
 - * Container [IcuLpit](#)
 - Parameter [IcuLpitModule](#)

Configuration Parameters

- Parameter [IcuLpitDebugEnabled](#)
- Parameter [IcuLpitDozeEnable](#)
- Container [IcuLpitChannels](#)
- * Container [IcuHwInterruptConfigList](#)
 - Parameter [IcuIsrHwId](#)
- Container [IcuGeneral](#)
 - * Parameter [IcuDevErrorDetect](#)
 - * Parameter [IcuEnableUserModeSupport](#)

4.2.1 Module Lpit_Icu

Configuration of ICU (Input Capture Unit) module.

Included containers:

- [IcuConfigSet](#)
- [IcuGeneral](#)

Property	Value
type	MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD;VARIANT-PRE-COMPILE

4.3 PORT_CI_ICU

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module [Port_Ci_Icu](#)
 - Container [IcuConfigSet](#)
 - * Parameter [IcuMaxChannel](#)
 - * Container [IcuChannel](#)
 - Parameter [IcuChannelId](#)
 - Parameter [IcuDefaultStartEdge](#)
 - Parameter [IcuMeasurementMode](#)
 - Parameter [IcuOverflowNotification](#)
 - Reference [IcuChannelRef](#)
 - Container [IcuSignalEdgeDetection](#)
 - Container [IcuSignalMeasurement](#)
 - Container [IcuTimestampMeasurement](#)
 - * Container [IcuPort](#)

- Parameter [IcuPortModule](#)
- Container [IcuPortChannels](#)
- * Container [IcuHwInterruptConfigList](#)
 - Parameter [IcuIsrHwId](#)
- Container [IcuGeneral](#)
 - * Parameter [IcuDevErrorDetect](#)
 - * Parameter [IcuEnableUserModeSupport](#)

4.3.1 Module Port_Ci_Icu

Configuration of ICU (Input Capture Unit) module.

Included containers:

- [IcuConfigSet](#)
- [IcuGeneral](#)

Property	Value
type	MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD;VARIANT-PRE-COMPILE

4.4 FTM_ICU

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module [Ftm_Icu](#)
 - Container [IcuConfigSet](#)
 - * Parameter [IcuMaxChannel](#)
 - * Container [IcuChannel](#)
 - Parameter [IcuChannelId](#)
 - Parameter [IcuDefaultStartEdge](#)
 - Parameter [IcuMeasurementMode](#)
 - Parameter [IcuOverflowNotification](#)
 - Reference [IcuChannelRef](#)
 - Container [IcuSignalEdgeDetection](#)
 - Container [IcuSignalMeasurement](#)
 - Container [IcuTimestampMeasurement](#)
 - * Container [IcuFtm](#)
 - Parameter [IcuFtmModule](#)

- Parameter [IcuFtmClockSource](#)
- Parameter [IcuFtmPrescaler](#)
- Parameter [IcuFtmPrescalerAlternate](#)
- Parameter [IcuFtmDebugMode](#)
- Parameter [IcuFtmModValue](#)
- Container [IcuFtmChannels](#)
- * Container [IcuHwInterruptConfigList](#)
 - Parameter [IcuIsrHwId](#)
- Container [IcuGeneral](#)
 - * Parameter [IcuDevErrorDetect](#)
 - * Parameter [IcuEnableUserModeSupport](#)

4.4.1 Module Ftm_Icu

Configuration of ICU (Input Capture Unit) module.

Included containers:

- [IcuConfigSet](#)
- [IcuGeneral](#)

Property	Value
type	MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD;VARIANT-PRE-COMPILE

4.5 LPTMR_ICU

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module [Lptmr_Icu](#)
 - Container [IcuConfigSet](#)
 - * Parameter [IcuMaxChannel](#)
 - * Container [IcuChannel](#)
 - Parameter [IcuChannelId](#)
 - Parameter [IcuDefaultStartEdge](#)
 - Parameter [IcuMeasurementMode](#)
 - Parameter [IcuOverflowNotification](#)
 - Reference [IcuChannelRef](#)

- Container [IcuSignalEdgeDetection](#)
- Container [IcuSignalMeasurement](#)
- Container [IcuTimestampMeasurement](#)
- * Container [IcuLptmr](#)
 - Parameter [IcuLptmrModule](#)
 - Parameter [PrescalerEnable](#)
 - Parameter [IcuLptmrPrescaler](#)
 - Parameter [IcuLptmrChannelClkSrc](#)
 - Parameter [IcuLptmrPinSelect](#)
 - Container [IcuLptmrChannels](#)
- * Container [IcuHwInterruptConfigList](#)
 - Parameter [IcuIsrHwId](#)
- Container [IcuGeneral](#)
 - * Parameter [IcuDevErrorDetect](#)
 - * Parameter [IcuEnableUserModeSupport](#)

4.5.1 Module Lptmr_Icu

Configuration of ICU (Input Capture Unit) module.

Included containers:

- [IcuConfigSet](#)
- [IcuGeneral](#)

Property	Value
type	MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD;VARIANT-PRE-COMPILE

4.6 CMP

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module [Cmp](#)
 - Container [IcuConfigSet](#)
 - * Parameter [IcuMaxChannel](#)
 - * Container [IcuChannel](#)
 - Parameter [IcuChannelId](#)
 - Parameter [IcuDefaultStartEdge](#)

Configuration Parameters

- Parameter [IcuMeasurementMode](#)
- Parameter [IcuOverflowNotification](#)
- Reference [IcuChannelRef](#)
- Container [IcuSignalEdgeDetection](#)
- Container [IcuSignalMeasurement](#)
- Container [IcuTimestampMeasurement](#)
- * Container [IcuLpCmp](#)
 - Parameter [IcuCmpInstanceNumber](#)
 - Container [IcuCmp](#)
 - Container [IcuDac](#)
 - Container [IcuTrigger](#)
- * Container [IcuHwInterruptConfigList](#)
 - Parameter [IcuIsrHwId](#)
- Container [IcuGeneral](#)
 - * Parameter [IcuDevErrorDetect](#)
 - * Parameter [IcuEnableUserModeSupport](#)

4.6.1 Module Cmp

Configuration of ICU (Input Capture Unit) module.

Included containers:

- [IcuConfigSet](#)
- [IcuGeneral](#)

Property	Value
type	MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD;VARIANT-PRE-COMPILE



Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

LPIT IPL	76
FTM IPL	83
LPTMR IPL	92
PORT_CI IPL	99
CMP IPL	102
Icu Driver	103

Chapter 6

Module Documentation

6.1 LPIT IPL

6.1.1 Detailed Description

LPIT HW module.

LPIT IP layer hardware module.

Data Structures

- struct [Lpit_Icu_Ip_ChannelConfigType](#)
LPIT channel configuration structure. [More...](#)
- struct [Lpit_Icu_Ip_ConfigType](#)
LPIT IP specific configuration structure type. [More...](#)
- struct [Lpit_Icu_Ip_ChannelsStateType](#)
LPIT channels state. [More...](#)

Macros

- `#define` [ICU_STOP_SEC_CODE](#)
LPIT_0 independent ISR declarations.

Types Reference

- typedef void(* [Lpit_Icu_Ip_NotifyType](#)) (void)
The notification functions shall have no parameters and no return value.
- typedef void(* [Lpit_Icu_Ip_CallbackType](#)) (uint16 callbackParam1, boolean callbackParam2)
Callback type for each channel.
- typedef void(* [Lpit_Icu_Ip_LogicChState](#)) (uint16 logicChannel, uint8 mask, boolean set)
Callback type for each channel.

Enum Reference

- enum [Lpit_Icu_Ip_StatusType](#)
Generic error codes.
- enum [Lpit_Icu_Ip_TimestampBufferType](#)

Type of operation for timestamp.

- enum [Lpit_Icu_Ip_MeasurementMode](#)

LPIT channel measurement mode supported.

Function Reference

- [Lpit_Icu_Ip_StatusType Lpit_Icu_Ip_Init](#) (uint8 instance, const [Lpit_Icu_Ip_ConfigType](#) *userConfig)
LPIT driver initialization function for LPIT module.
- void [Lpit_Icu_Ip_EnableInterrupt](#) (uint8 instance, uint8 hwChannel)
LPIT driver function that enables interrupts on a LPIT channel.
- void [Lpit_Icu_Ip_DisableInterrupt](#) (uint8 instance, uint8 hwChannel)
LPIT driver function that disables interrupts on a LPIT channel.
- void [Lpit_Icu_Ip_EnableNotification](#) (uint8 instance, uint8 hwChannel)
Driver function Enable Notification for timestamp.
- void [Lpit_Icu_Ip_DisableNotification](#) (uint8 instance, uint8 hwChannel)
Driver function Disable Notification for timestamp.
- uint16 [Lpit_Icu_Ip_GetTimestampIndex](#) (uint8 instance, uint8 channel)
Get timestamp index for timestamp mode.

6.1.2 Data Structure Documentation

6.1.2.1 struct [Lpit_Icu_Ip_ChannelConfigType](#)

LPIT channel configuration structure.

Definition at line 128 of file [Lpit_Icu_Ip_Types.h](#).

Data Fields

const uint8	hwChannel	Channel hardware index.
const uint8	triggerSelect	Trigger to use for starting and/or reloading the LPIT timer.
const uint8	triggerSource	Select source of trigger.
const Lpit_Icu_Ip_TimestampBufferType	timestampBuffer	
Lpit_Icu_Ip_CallbackType	callback	The callback function for channels edge detect events.
uint8	callbackParams	The parameters of callback functions for channels events.
Lpit_Icu_Ip_LogicChState	logicChStateCallback	Store address of function used to change the logic state of the channel in HLD.
const Lpit_Icu_Ip_NotifyType	lpitChannelNotify	The notification functions shall have no parameters and no return value.

6.1.2.2 struct [Lpit_Icu_Ip_ConfigType](#)

LPIT IP specific configuration structure type.

Definition at line 143 of file [Lpit_Icu_Ip_Types.h](#).

Data Fields

const uint8	instance	Instance hardware index.
const boolean	debugState	Debug(freeze) mode.
const boolean	dozeMode	enable/disable Doze mode.
const uint8	numberOfChannels	Number of LPIT channels on the current instance.
const Lpit_Icu_Ip_ChannelConfigType (*	pChannelsConfig[]	Pointer to the array of configured channels.

6.1.2.3 struct Lpit_Icu_Ip_ChannelsStateType

LPIT channels state.

Definition at line 161 of file Lpit_Icu_Ip_Types.h.

Data Fields

boolean	initState	Initialization status.
Lpit_Icu_Ip_MeasurementMode	measurementMode	Measurement mode for current channel.
Lpit_Icu_Ip_CallbackType	callback	The callback function for channels edge detect events.
uint8	callbackParams	The parameters of callback functions for channels events.
Lpit_Icu_Ip_LogicChState	logicChStateCallback	Store address of function used to change the logic state of the channel in HLD.
Lpit_Icu_Ip_NotifyType	lpitChannelNotify	The notification functions shall have no parameters and no return value.
boolean	notificationEnable	Store the initialization state that determines whether Notifications are enabled, it is always TRUE with standalone IPL and FALSE with AUTOSAR mode.

6.1.3 Macro Definition Documentation

6.1.3.1 ICU_STOP_SEC_CODE

```
#define ICU_STOP_SEC_CODE
```

LPIT_0 independent ISR declarations.

Definition at line 119 of file Lpit_Icu_Ip_Irq.h.

6.1.4 Types Reference

6.1.4.1 Lpit_Icu_Ip_NotifyType

```
typedef void(* Lpit_Icu_Ip_NotifyType) (void)
```

The notification functions shall have no parameters and no return value.

Definition at line 99 of file Lpit_Icu_Ip_Types.h.

6.1.4.2 Lpit_Icu_Ip_CallbackType

```
typedef void(* Lpit_Icu_Ip_CallbackType) (uint16 callbackParam1, boolean callbackParam2)
```

Callback type for each channel.

Definition at line 102 of file Lpit_Icu_Ip_Types.h.

6.1.4.3 Lpit_Icu_Ip_LogicChState

```
typedef void(* Lpit_Icu_Ip_LogicChState) (uint16 logicChannel, uint8 mask, boolean set)
```

Callback type for each channel.

Definition at line 105 of file Lpit_Icu_Ip_Types.h.

6.1.5 Enum Reference

6.1.5.1 Lpit_Icu_Ip_StatusType

```
enum Lpit_Icu_Ip_StatusType
```

Generic error codes.

Enumerator

LPIT_IP_STATUS_SUCCESS	Generic operation success status.
LPIT_IP_STATUS_ERROR	Generic operation failure status.

Definition at line 88 of file Lpit_Icu_Ip_Types.h.

6.1.5.2 Lpit_Icu_Ip_TimestampBufferType

```
enum Lpit_Icu_Ip_TimestampBufferType
```

Type of operation for timestamp.

Enumerator

LPIT_ICU_IP_NO_BUFFER	No timestamp.
LPIT_ICU_IP_CIRCULAR_BUFFER	The timestamp with circular buffer .
LPIT_ICU_IP_LINEAR_BUFFER	The timestamp with linear buffer .

Definition at line 109 of file Lpit_Icu_Ip_Types.h.

6.1.5.3 Lpit_Icu_Ip_MeasurementMode

enum `Lpit_Icu_Ip_MeasurementMode`

LPIT channel measurement mode supported.

Enumerator

<code>LPIT_ICU_MODE_NO_MEASUREMENT</code>	No measurement mode.
<code>LPIT_ICU_MODE_SIGNAL_EDGE_DETECT</code>	Signal edge detect measurement mode.
<code>LPIT_ICU_MODE_TIMESTAMP</code>	Timestamp measurement mode.

Definition at line 153 of file `Lpit_Icu_Ip_Types.h`.

6.1.6 Function Reference

6.1.6.1 Lpit_Icu_Ip_Init()

```
Lpit_Icu_Ip_StatusType Lpit_Icu_Ip_Init (
    uint8 instance,
    const Lpit_Icu_Ip_ConfigType * userConfig )
```

LPIT driver initialization function for LPIT module.

This function is called separately for each LPIT instace and will do the following:

- enables the LPIT module
- configures the debug mode (enabled or disabled)
- disable the IRQ corresponding to the LPIT channel
- clears the (pending) interrupt flag corresponding to LPIT channel
- enable channel interrupts
- Set Trigger Input Capture Mode

Parameters

in	<i>instance</i>	- hardware instance to be configured
in	<i>userConfig</i>	- configuration of the instance that will be intialized

Returns

`Lpit_Icu_Ip_StatusType`

6.1.6.2 Lpit_Icu_Ip_EnableInterrupt()

```
void Lpit_Icu_Ip_EnableInterrupt (
    uint8 instance,
    uint8 hwChannel )
```

LPIT driver function that enables interrupts on a LPIT channel.

This function enables LPIT channel interrupt.

Parameters

in	<i>instance</i>	- hardware instance of the module
in	<i>hwChannel</i>	- channel instance of the module

6.1.6.3 Lpit_Icu_Ip_DisableInterrupt()

```
void Lpit_Icu_Ip_DisableInterrupt (
    uint8 instance,
    uint8 hwChannel )
```

LPIT driver function that disables interrupts on a LPIT channel.

This function disables LPIT channel interrupt.

Parameters

in	<i>instance</i>	- hardware instance of the module
in	<i>hwChannel</i>	- channel instance of the module

6.1.6.4 Lpit_Icu_Ip_EnableNotification()

```
void Lpit_Icu_Ip_EnableNotification (
    uint8 instance,
    uint8 hwChannel )
```

Driver function Enable Notification for timestamp.

Parameters

in	<i>instance</i>	Hardware instance of FTM used.
in	<i>hwChannel</i>	Hardware channel of FTM used.

Returns

void

6.1.6.5 Lpit_Icu_Ip_DisableNotification()

```
void Lpit_Icu_Ip_DisableNotification (
    uint8 instance,
    uint8 hwChannel )
```

Driver function Disable Notification for timestamp.

Parameters

in	<i>instance</i>	Hardware instance of FTM used.
in	<i>hwChannel</i>	Hardware channel of FTM used.

Returns

void

6.1.6.6 Lpit_Icu_Ip_GetTimestampIndex()

```
uint16 Lpit_Icu_Ip_GetTimestampIndex (
    uint8 instance,
    uint8 channel )
```

Get timestamp index for timestamp mode.

Parameters

in	<i>instance</i>	Hardware instance of FTM used.
in	<i>hwChannel</i>	Hardware channel of FTM used.

Returns

uint16

6.2 FTM IPL

6.2.1 Detailed Description

FTM HW module.

FTM IP layer hardware module.

Data Structures

- struct [Ftm_Icu_Ip_DutyCycleType](#)
Structure that contains ICU Duty cycle parameters. It contains the values needed for calculating duty cycles i.e Period time value and active time value. [More...](#)
- struct [Ftm_Icu_Ip_ChannelConfigType](#)
FlexTimer driver Input capture parameters for each channel. [More...](#)
- struct [Ftm_Icu_Ip_InstanceConfigType](#)
FTM IP layer module configuration. [More...](#)
- struct [Ftm_Icu_Ip_ConfigType](#)
FTM driver input capture parameters. [More...](#)
- struct [Ftm_Icu_Ip_ChStateType](#)
This structure is used by the IPL driver for internal logic. [More...](#)
- struct [Ftm_Icu_Ip_InstStateType](#)
This structure is used by the IPL driver for internal logic. [More...](#)

Macros

- `#define` [CHAN0_IDX](#)
Channel number for CHAN0.
- `#define` [CHAN1_IDX](#)
Channel number for CHAN1.
- `#define` [CHAN2_IDX](#)
Channel number for CHAN2.
- `#define` [CHAN3_IDX](#)
Channel number for CHAN3.
- `#define` [CHAN4_IDX](#)
Channel number for CHAN4.
- `#define` [FTM_MAX_VAL_COUNTER](#)
Maximum value of FTM counter.
- `#define` [FTM_COMBINE_COMBINEx_SHIFT](#)(u8ChannelIdx)
Set flag COMINEx for specified pair: 0, 1, 2.

Types Reference

- typedef void(* [Ftm_Icu_Ip_NotifyType](#)) (void)
The notification functions shall have no parameters and no return value.
- typedef void(* [Ftm_Icu_Ip_CallbackType](#)) (uint16 callbackParam1, boolean callbackParam2)

Callback type for each channel.

- typedef void(* [Ftm_Icu_Ip_LogicChState](#)) (uint16 logicChannel, uint8 mask, boolean set)

Callback type for each channel.

Enum Reference

- enum [Ftm_Icu_Ip_ClockSourceType](#)
FTM clock source selection.
- enum [Ftm_Icu_Ip_DebugModeType](#)
FTM debug modes of operation.
- enum [Ftm_Icu_Ip_EdgeType](#)
Activation condition for the measurement - selecting edge type.
- enum [Ftm_Icu_Ip_ModeType](#)
Operation mode for ICU driver.
- enum [Ftm_Icu_Ip_SubModeType](#)
Enable/disable DMA support for timestamp.
- enum [Ftm_Icu_Ip_MeasType](#)
Type of operation for signal measurement.
- enum [Ftm_Icu_Ip_LevelType](#)
Enumeration used for returning the level of input pin.
- enum [Ftm_Icu_Ip_ClockModeType](#)
Definition of prescaler type (Normal or Alternate)
- enum [Ftm_Icu_Ip_StatusType](#)
Generic error codes.
- enum [Ftm_Icu_Ip_TimestampBufferType](#)
Type of operation for timestamp.

6.2.2 Data Structure Documentation

6.2.2.1 struct [Ftm_Icu_Ip_DutyCycleType](#)

Structure that contains ICU Duty cycle parameters. It contains the values needed for calculating duty cycles i.e Period time value and active time value.

Definition at line 244 of file [Ftm_Icu_Ip_Types.h](#).

Data Fields

uint16	ActiveTime	Low or High time value.
uint16	PeriodTime	Period time value.

6.2.2.2 struct [Ftm_Icu_Ip_ChannelConfigType](#)

FlexTimer driver Input capture parameters for each channel.

Definition at line 257 of file [Ftm_Icu_Ip_Types.h](#).

Data Fields

uint8	hwChannel	Physical hardware channel ID
Ftm_Icu_Ip_ModeType	chMode	FlexTimer module mode of operation
Ftm_Icu_Ip_SubModeType	chSubMode	FlexTimer specific name of operation to execute
Ftm_Icu_Ip_MeasType	measurementMode	Measurement Mode for signal measurement
Ftm_Icu_Ip_EdgeType	edgeAlignement	Edge alignment Mode for signal measurement
boolean	continuouseEn	Continuous measurement state
uint8	filterValue	Filter Value
Ftm_Icu_Ip_CallbackType	callback	The callback function for channels edge detect events
uint8	callbackParams	The parameters of callback functions for channels events
Ftm_Icu_Ip_TimestampBufferType	timestampBufferType	Timestamp buffer type for timestamp mode
Ftm_Icu_Ip_LogicChState	logicChStateCallback	Store address of function used to change the logic state of the channel in HLD.
Ftm_Icu_Ip_NotifyType	ftmChannelNotification	The notification functions shall have no parameters and no return value.
Ftm_Icu_Ip_NotifyType	ftmOverflowNotification	The overflow notification functions shall have no parameters and no return value.

6.2.2.3 struct Ftm_Icu_Ip_InstanceConfigType

FTM IP layer module configuration.

Definition at line 277 of file Ftm_Icu_Ip_Types.h.

Data Fields

Ftm_Icu_Ip_ClockSourceType	cfgClkSrc	Type of clock source used.
uint8	cfgPrescaler	Prescaler value.
uint8	cfgAltPrescaler	Alternant prescaler value.
Ftm_Icu_Ip_DebugModeType	debugMode	Debug mode.
uint16	maxCountValue	Maximum counter value. Minimum value is 0.

6.2.2.4 struct Ftm_Icu_Ip_ConfigType

FTM driver input capture parameters.

Definition at line 294 of file Ftm_Icu_Ip_Types.h.

Data Fields

uint8	nNumChannels	Number of input capture channel used.
const Ftm_Icu_Ip_InstanceConfigType *	pInstanceConfig	Input capture instance configuration.
const Ftm_Icu_Ip_ChannelConfigType (*	pChannelsConfig)[]	Input capture channels configuration.

6.2.2.5 struct Ftm_Icu_Ip_ChStateType

This structure is used by the IPL driver for internal logic.

Definition at line 306 of file Ftm_Icu_Ip_Types.h.

Data Fields

boolean	firstEdge	Store the status of the first measurement.
boolean	firstEdgePolarity	Store the first edge come to measurement in BOTH_EDGE mode.
Ftm_Icu_Ip_MeasType	measurement	Signal measurement mode.
uint16	ftm_Icu_Ip_aPeriod	Variable for saving the period.
uint16	ftm_Icu_Ip_aActivePulseWidth	Variable for saving the pulse width of active time.
Ftm_Icu_Ip_NotifyType	ftmChannelNotification	The notification functions for TIME_STAMP or SIGNAL_EDGE_DETECT mode.
Ftm_Icu_Ip_NotifyType	ftmOverflowNotification	The overflow notification functions.
uint16	edgeNumbers	Logic variable to count edges.
Ftm_Icu_Ip_ModeType	channelMode	FTM channel mode.
Ftm_Icu_Ip_EdgeType	edgeTrigger	Type of edge used for activation.
Ftm_Icu_Ip_SubModeType	dmaMode	Support DMA or not.
Ftm_Icu_ValueType *	ftm_Icu_Ip_aBuffer	Pointer to the buffer-array where the timestamp values shall be placed.
uint16	ftm_Icu_Ip_aBufferSize	Variable for saving the size of the external buffer (number of entries).
uint16	ftm_Icu_Ip_aBufferNotify	Variable for saving Notification interval (number of events).
uint16	ftm_Icu_Ip_aNotifyCount	Variable for saving the number of notify counts.
uint16	ftm_Icu_Ip_aBufferIndex	Variable for saving the time stamp index.
Ftm_Icu_Ip_TimestampBufferType	timestampBufferType	Timestamp buffer type for timestamp mode.
Ftm_Icu_Ip_LogicChState	logicChStateCallback	Store address of function used to change the logic state of the channel in HLD.
Ftm_Icu_Ip_CallbackType	callback	Calback for other types of measurement.
uint16	callbackParam	Logic channel for which callback is executed.
boolean	notificationEnable	Store the state determines whether Notifications are enabled or not.

6.2.2.6 struct Ftm_Icu_Ip_InstStateType

This structure is used by the IPL driver for internal logic.

Definition at line 358 of file Ftm_Icu_Ip_Types.h.

Data Fields

boolean	instInit	Module initialization state.
uint8	prescaler	Module prescaler value.
uint8	prescalerAlternate	Module alternate prescaler value.
uint8	spuriousMask	

6.2.3 Macro Definition Documentation

6.2.3.1 CHAN0_IDX

```
#define CHAN0_IDX
```

Channel number for CHAN0.

Definition at line 91 of file Ftm_Icu_Ip_Types.h.

6.2.3.2 CHAN1_IDX

```
#define CHAN1_IDX
```

Channel number for CHAN1.

Definition at line 93 of file Ftm_Icu_Ip_Types.h.

6.2.3.3 CHAN2_IDX

```
#define CHAN2_IDX
```

Channel number for CHAN2.

Definition at line 95 of file Ftm_Icu_Ip_Types.h.

6.2.3.4 CHAN3_IDX

```
#define CHAN3_IDX
```

Channel number for CHAN3.

Definition at line 97 of file Ftm_Icu_Ip_Types.h.

6.2.3.5 CHAN4_IDX

```
#define CHAN4_IDX
```

Channel number for CHAN4.

Definition at line 99 of file Ftm_Icu_Ip_Types.h.

6.2.3.6 FTM_MAX_VAL_COUNTER

```
#define FTM_MAX_VAL_COUNTER
```

Maximum value of FTM counter.

Definition at line 102 of file Ftm_Icu_Ip_Types.h.

6.2.3.7 FTM_COMBINE_COMBINE_x_SHIFT

```
#define FTM_COMBINE_COMBINEx_SHIFT(  
    u8ChannelIdx )
```

Set flag COMINEx for specified pair: 0, 1, 2.

Definition at line 106 of file Ftm_Icu_Ip_Types.h.

6.2.4 Types Reference

6.2.4.1 Ftm_Icu_Ip_NotifyType

```
typedef void(* Ftm_Icu_Ip_NotifyType) (void)
```

The notification functions shall have no parameters and no return value.

Definition at line 239 of file Ftm_Icu_Ip_Types.h.

6.2.4.2 Ftm_Icu_Ip_CallbackType

```
typedef void(* Ftm_Icu_Ip_CallbackType) (uint16 callbackParam1, boolean callbackParam2)
```

Callback type for each channel.

Definition at line 251 of file Ftm_Icu_Ip_Types.h.

6.2.4.3 Ftm_Icu_Ip_LogicChState

```
typedef void(* Ftm_Icu_Ip_LogicChState) (uint16 logicChannel, uint8 mask, boolean set)
```

Callback type for each channel.

Definition at line 254 of file Ftm_Icu_Ip_Types.h.

6.2.5 Enum Reference

6.2.5.1 Ftm_Icu_Ip_ClockSourceType

```
enum Ftm_Icu_Ip_ClockSourceType
```

FTM clock source selection.

Enumerator

FTM_NO_CLOCK_SELECTED	No clock selected. This in effect disables the FTM counter.
-----------------------	---

Enumerator

FTM_SYSTEM_CLOCK	FTM input clock.
FTM_FIXED_FREQUENCY_CLOCK	Fixed frequency clock.
FTM_EXTERNAL_CLOCK	External clock.

Definition at line 114 of file Ftm_Icu_Ip_Types.h.

6.2.5.2 Ftm_Icu_Ip_DebugModeType

enum [Ftm_Icu_Ip_DebugModeType](#)

FTM debug modes of operation.

Enumerator

MODE↔ _0	FTM counter - stopped.
MODE↔ _1	FTM counter - stopped.
MODE↔ _2	FTM counter - stopped.
MODE↔ _3	FTM counter - functional mode.

Definition at line 127 of file Ftm_Icu_Ip_Types.h.

6.2.5.3 Ftm_Icu_Ip_EdgeType

enum [Ftm_Icu_Ip_EdgeType](#)

Activation condition for the measurement - selecting edge type.

Enumerator

FTM_ICU_NO_PIN_CONTROL	No trigger.
FTM_ICU_RISING_EDGE	Rising edge trigger.
FTM_ICU_FALLING_EDGE	Falling edge trigger.
FTM_ICU_BOTH_EDGES	Rising and falling edge trigger.

Definition at line 140 of file Ftm_Icu_Ip_Types.h.

6.2.5.4 Ftm_Icu_Ip_ModeType

enum [Ftm_Icu_Ip_ModeType](#)

Operation mode for ICU driver.

Enumerator

FTM_ICU_MODE_NO_MEASUREMENT	No measurement mode.
FTM_ICU_MODE_SIGNAL_EDGE_DETECT	Signal edge detect measurement mode.
FTM_ICU_MODE_SIGNAL_MEASUREMENT	Signal measurement mode.
FTM_ICU_MODE_TIMESTAMP	Timestamp measurement mode.
FTM_ICU_MODE_EDGE_COUNTER	Edge counter measurement mode.

Definition at line 153 of file Ftm_Icu_Ip_Types.h.

6.2.5.5 Ftm_Icu_Ip_SubModeType

enum [Ftm_Icu_Ip_SubModeType](#)

Enable/disable DMA support for timestamp.

Definition at line 168 of file Ftm_Icu_Ip_Types.h.

6.2.5.6 Ftm_Icu_Ip_MeasType

enum [Ftm_Icu_Ip_MeasType](#)

Type of operation for signal measurement.

Enumerator

FTM_ICU_NO_MEASUREMENT	No measurement.
FTM_ICU_LOW_TIME	The time measurement for OFF period.
FTM_ICU_HIGH_TIME	The time measurement for ON period.
FTM_ICU_PERIOD_TIME	Period measurement between two consecutive falling/raising edges.
FTM_ICU_DUTY_CYCLE	The fraction of active period.

Definition at line 177 of file Ftm_Icu_Ip_Types.h.

6.2.5.7 Ftm_Icu_Ip_LevelType

enum [Ftm_Icu_Ip_LevelType](#)

Enumeration used for returning the level of input pin.

Enumerator

FTM_ICU_LEVEL_LOW	Low level state.
FTM_ICU_LEVEL_HIGH	High level state.

Definition at line 194 of file Ftm_Icu_Ip_Types.h.

6.2.5.8 Ftm_Icu_Ip_ClockModeType

enum `Ftm_Icu_Ip_ClockModeType`

Definition of prescaler type (Normal or Alternate)

Enumerator

FTM_ICU_NORMAL_CLK	Normal prescaler
FTM_ICU_ALTERNATE_CLK	Alternate prescaler

Definition at line 205 of file `Ftm_Icu_Ip_Types.h`.

6.2.5.9 Ftm_Icu_Ip_StatusType

enum `Ftm_Icu_Ip_StatusType`

Generic error codes.

Enumerator

FTM_IP_STATUS_SUCCESS	Generic operation success status.
FTM_IP_STATUS_ERROR	Generic operation failure status.

Definition at line 213 of file `Ftm_Icu_Ip_Types.h`.

6.2.5.10 Ftm_Icu_Ip_TimestampBufferType

enum `Ftm_Icu_Ip_TimestampBufferType`

Type of operation for timestamp.

Enumerator

FTM_ICU_NO_TIMESTAMP	No timestamp.
FTM_ICU_CIRCULAR_BUFFER	The timestamp with circular buffer .
FTM_ICU_LINEAR_BUFFER	The timestamp with linear buffer .

Definition at line 224 of file `Ftm_Icu_Ip_Types.h`.

6.3 LPTMR IPL

6.3.1 Detailed Description

LPTMR HW module.

LPTMR IP layer hardware module.

Data Structures

- struct [Lptmr_Icu_Ip_ChannelConfigType](#)
Lptmr Channel specific configuration structure type. [More...](#)
- struct [Lptmr_Icu_Ip_ConfigType](#)
Lptmr IP specific configuration structure type. [More...](#)

Macros

- `#define ICU_STOP_SEC_CODE`
LPTRM_0 Channel 0 independent ISR declarations.
- `#define LPTMR_ICU_IP_TM_MODE`
Pulse Time Counter mode.

Types Reference

- typedef void(* [Lptmr_Icu_Ip_NotifyType](#)) (void)
The notification functions shall have no parameters and no return value.
- typedef void(* [Lptmr_Icu_Ip_CallbackType](#)) (uint16 logicChannel, boolean overflow)
Callback type for each channel.

Enum Reference

- enum [Lptmr_Icu_PrescalerType](#)
Prescaler Selection.
- enum [Lptmr_Icu_MeasurementModeType](#)
Lptmr_Icu_ChannelMeasurementModeType.
- enum [Lptmr_Icu_Ip_PinSelectType](#)
Definition of input pin type.
- enum [Lptmr_Icu_Ip_ClockSourceType](#)
LPTMR clock source selection.
- enum [Lptmr_Icu_Ip_EdgeType](#)
Activation condition for the measurement - selecting edge type.
- enum [Lptmr_Icu_Ip_StatusType](#)
Generic error codes.

Function Reference

- [Lptmr_Icu_Ip_StatusType Lptmr_Icu_Ip_Init](#) (uint8 instance, const [Lptmr_Icu_Ip_ConfigType](#) *user←↵ Config)
- void [Lptmr_Icu_Ip_SetActivationCondition](#) (uint8 instance, [Lptmr_Icu_Ip_EdgeType](#) activation)
This function enables the requested activation condition(rising, falling or both edges) for corresponding LPTMR channels.
- void [Lptmr_Icu_Ip_EnableNotification](#) (uint8 instance)
Driver function Enable Notification for timestamp.
- void [Lptmr_Icu_Ip_DisableNotification](#) (uint8 instance)
Driver function Disable Notification for timestamp.

6.3.2 Data Structure Documentation

6.3.2.1 struct Lptmr_Icu_Ip_ChannelConfigType

Lptmr Channel specific configuration structure type.

Definition at line 197 of file Lptmr_Icu_Ip_Types.h.

Data Fields

uint8	HwChannel	Physical hardware channel ID
const Lptmr_Icu_Ip_EdgeType	DefaultStartEdge	Lptmr Default Start Edge
const Lptmr_Icu_MeasurementModeType	MeasurementModeType	Lptmr MeasurementMode
Lptmr_Icu_Ip_NotifyType	lptmrChannelNotification	The notification functions shall have no parameters and no return value.
Lptmr_Icu_Ip_CallbackType	callback	interrupt callback function.
uint16	callbackParam	The parameters of callback functions for channels events

6.3.2.2 struct Lptmr_Icu_Ip_ConfigType

Lptmr IP specific configuration structure type.

Definition at line 210 of file Lptmr_Icu_Ip_Types.h.

Data Fields

uint8	nNumChannels	Number of Lptmr channels in the Icu configuration
const Lptmr_Icu_Ip_PinSelectType	PinSelect	Lptmr channel parameters
const Lptmr_Icu_Ip_ClockSourceType	ClockSource	Lptmr clock source
const Lptmr_Icu_PrescalerType	Prescaler	The Lptmr Prescaler values

Data Fields

boolean	PrescalerEnable	The Lptmr Prescaler Bypass
const Lptmr_Icu_Ip_ChannelConfigType (*	pChannelsConfig[]	Pointer to the configured channels for Lptmr

6.3.3 Macro Definition Documentation

6.3.3.1 ICU_STOP_SEC_CODE

```
#define ICU_STOP_SEC_CODE
```

LPTRM_0 Channel 0 independent ISR declarations.

Definition at line 107 of file Lptmr_Icu_Ip_Irq.h.

6.3.3.2 LPTMR_ICU_IP_TM_MODE

```
#define LPTMR_ICU_IP_TM_MODE
```

Pulse Time Counter mode.

Definition at line 89 of file Lptmr_Icu_Ip_Types.h.

6.3.4 Types Reference

6.3.4.1 Lptmr_Icu_Ip_NotifyType

```
typedef void(* Lptmr_Icu_Ip_NotifyType) (void)
```

The notification functions shall have no parameters and no return value.

Definition at line 176 of file Lptmr_Icu_Ip_Types.h.

6.3.4.2 Lptmr_Icu_Ip_CallbackType

```
typedef void(* Lptmr_Icu_Ip_CallbackType) (uint16 logicChannel, boolean overflow)
```

Callback type for each channel.

Definition at line 179 of file Lptmr_Icu_Ip_Types.h.

6.3.5 Enum Reference

6.3.5.1 Lptmr_Icu_PrescalerType

enum `Lptmr_Icu_PrescalerType`

Prescaler Selection.

Enumerator

LPTMR_ICU_GLITCH_FILTER_2	Timer mode: prescaler 2, Glitch filter mode: invalid
LPTMR_ICU_GLITCH_FILTER_4	Timer mode: prescaler 4, Glitch filter mode: 2 clocks
LPTMR_ICU_GLITCH_FILTER_8	Timer mode: prescaler 8, Glitch filter mode: 4 clocks
LPTMR_ICU_GLITCH_FILTER_16	Timer mode: prescaler 16, Glitch filter mode: 8 clocks
LPTMR_ICU_GLITCH_FILTER_32	Timer mode: prescaler 32, Glitch filter mode: 16 clocks
LPTMR_ICU_GLITCH_FILTER_64	Timer mode: prescaler 64, Glitch filter mode: 32 clocks
LPTMR_ICU_GLITCH_FILTER_128	Timer mode: prescaler 128, Glitch filter mode: 64 clocks
LPTMR_ICU_GLITCH_FILTER_256	Timer mode: prescaler 256, Glitch filter mode: 128 clocks
LPTMR_ICU_GLITCH_FILTER_512	Timer mode: prescaler 512, Glitch filter mode: 256 clocks
LPTMR_ICU_GLITCH_FILTER_1024	Timer mode: prescaler 1024, Glitch filter mode: 512 clocks
LPTMR_ICU_GLITCH_FILTER_2048	Timer mode: prescaler 2048, Glitch filter mode: 1024 clocks
LPTMR_ICU_GLITCH_FILTER_4096	Timer mode: prescaler 4096, Glitch filter mode: 2048 clocks
LPTMR_ICU_GLITCH_FILTER_8192	Timer mode: prescaler 8192, Glitch filter mode: 4096 clocks
LPTMR_ICU_GLITCH_FILTER_16384	Timer mode: prescaler 16384, Glitch filter mode: 8192 clocks
LPTMR_ICU_GLITCH_FILTER_32768	Timer mode: prescaler 32768, Glitch filter mode: 16384 clocks
LPTMR_ICU_GLITCH_FILTER_65536	Timer mode: prescaler 65536, Glitch filter mode: 32768 clocks

Definition at line 97 of file Lptmr_Icu_Ip_Types.h.

6.3.5.2 Lptmr_Icu_MeasurementModeType

enum `Lptmr_Icu_MeasurementModeType`

Lptmr_Icu_ChannelMeasurementModeType.

Type that indicates the channel mode type(capture mode, edge counter).

Definition at line 121 of file Lptmr_Icu_Ip_Types.h.

6.3.5.3 Lptmr_Icu_Ip_PinSelectType

enum `Lptmr_Icu_Ip_PinSelectType`

Definition of input pin type.

Enumerator

TRGMUX_OUTPUT	TRGMUX output.
ALT1	LPTMR_ALT1 pin.
ALT2	LPTMR_ALT2 pin.
ALT3	LPTMR_ALT3 pin.

Definition at line 132 of file Lptmr_Icu_Ip_Types.h.

6.3.5.4 Lptmr_Icu_Ip_ClockSourceType

enum `Lptmr_Icu_Ip_ClockSourceType`

LPTMR clock source selection.

Enumerator

<code>LPTMR_ICU_SIRCDIV2_CLK</code>	Select SIRCDIV2_CLK.
<code>LPTMR_ICU_LPO1K_CLK</code>	Select LPO1K_CLK.
<code>LPTMR_ICU_RTC_CLK</code>	Select RTC_CLK.
<code>LPTMR_ICU_PCC_LPTMR0</code>	Select PCC_LPTMR0.

Definition at line 145 of file `Lptmr_Icu_Ip_Types.h`.

6.3.5.5 Lptmr_Icu_Ip_EdgeType

enum `Lptmr_Icu_Ip_EdgeType`

Activation condition for the measurement - selecting edge type.

Enumerator

<code>LPTMR_ICU_RISING_EDGE</code>	Rising edge trigger.
<code>LPTMR_ICU_FALLING_EDGE</code>	Rising edge trigger.

Definition at line 156 of file `Lptmr_Icu_Ip_Types.h`.

6.3.5.6 Lptmr_Icu_Ip_StatusType

enum `Lptmr_Icu_Ip_StatusType`

Generic error codes.

Enumerator

<code>LPTMR_IP_STATUS_SUCCESS</code>	Generic operation success status.
<code>LPTMR_IP_STATUS_ERROR</code>	Generic operation failure status.

Definition at line 165 of file `Lptmr_Icu_Ip_Types.h`.

6.3.6 Function Reference

6.3.6.1 Lptmr_Icu_Ip_Init()

```
Lptmr_Icu_Ip_StatusType Lptmr_Icu_Ip_Init (
    uint8 instance,
    const Lptmr_Icu_Ip_ConfigType * userConfig )
```

Parameters

in	<i>instance</i>	- Hardware instance of LPTMR used.
in	<i>param</i>	- Configuration of the input capture channel.

Returns

void

6.3.6.2 Lptmr_Icu_Ip_SetActivationCondition()

```
void Lptmr_Icu_Ip_SetActivationCondition (
    uint8 instance,
    Lptmr_Icu_Ip_EdgeType activation )
```

This function enables the requested activation condition(rising, falling or both edges) for corresponding LPTMR channels.

Parameters

in	<i>instance</i>	- Hardware instance of LPTMR used.
in	<i>activation</i>	Edge activation type used. - LPTMR_ICU_RISING_EDGE : count pulse on Rising Edge - LPTMR_ICU_FALLING_EDGE: count pulse on Falling Edge

Returns

void

6.3.6.3 Lptmr_Icu_Ip_EnableNotification()

```
void Lptmr_Icu_Ip_EnableNotification (
    uint8 instance )
```

Driver function Enable Notification for timestamp.

6.3.6.4 Lptmr_Icu_Ip_DisableNotification()

```
void Lptmr_Icu_Ip_DisableNotification (
    uint8 instance )
```

Driver function Disable Notification for timestamp.

6.4 PORT_CI IPL

6.4.1 Detailed Description

PORT_CI HW module.

PORT_CI IP layer hardware module.

PORT_CI provides control over all electrical pin controls and ports with 16 bits of bidirectional, general-purpose input and output signals. PORT_CI enables you to select the functions and electrical characteristics that appear on external chip pins. It also controls the multiplexing of internal signals from one module to another and controls chip I/O. It supports as many as 32 external interrupts with trigger event configuration.

PORT_CI provides dedicated pad control to general-purpose pads that can be configured as either inputs or outputs. It provides registers for you to read values from GPIO pads configured as inputs and to write values to GPIO pads configured as outputs:

- When configured as output, you can write to an internal register to control the state driven on the associated output pad.
- When configured as **input**, you can detect the state of the associated pad by reading the value from an internal register.
- When configured as input and output, the pad value can be read back to check if the written value appeared on the pad.

Data Structures

- struct [Port_Ci_Icu_Ip_ConfigType](#)
Port_Ci IP specific configuration structure type. [More...](#)
- struct [Port_Ci_Icu_Ip_State](#)
PORT CI IP state structure. [More...](#)

Types Reference

- typedef void(* [Port_Ci_Icu_Ip_NotifyType](#)) (void)
The notification functions shall have no parameters and no return value.
- typedef void(* [Port_Ci_Icu_Ip_CallbackType](#)) (uint16 callbackParam1, boolean callbackParam2)
Callback signature used in each channel with an active interrupt.

Enum Reference

- enum [Port_Ci_Icu_Ip_EdgeType](#)
Port_Ci_Icu_Ip_EdgeType.
- enum [Port_Ci_Icu_Ip_StatusType](#)
Generic error codes.

Function Reference

- void [Port_Ci_Icu_Ip_EnableNotification](#) (uint8 instance, uint8 hwChannel)
Driver function Enable Notification for timestamp.
- void [Port_Ci_Icu_Ip_DisableNotification](#) (uint8 instance, uint8 hwChannel)
Driver function Disable Notification for timestamp.

6.4.2 Data Structure Documentation

6.4.2.1 struct Port_Ci_Icu_Ip_ConfigType

Port_Ci IP specific configuration structure type.

Definition at line 149 of file Port_Ci_Icu_Ip_Types.h.

Data Fields

uint8	nNumChannels	
const Port_Ci_Icu_Ip_ChannelConfigType(*	pChannelsConfig[]	Number of Port_Ci channels in the Icu configuration.

6.4.2.2 struct Port_Ci_Icu_Ip_State

PORT CI IP state structure.

This structure is used by the IPL driver for internal logic. The content is populated at initialization time.

Definition at line 160 of file Port_Ci_Icu_Ip_Types.h.

Data Fields

boolean	chInit	
Port_Ci_Icu_Ip_CallbackType	callback	Initialization state.
Port_Ci_Icu_Ip_NotifyType	PortCiChannelNotification	Pointer to the callback function.
uint16	callbackParam	The notification functions for SIGNAL_EDGE_DETECT mode.
boolean	notificationEnable	The logic channel for which callback is set.

6.4.3 Types Reference

6.4.3.1 Port_Ci_Icu_Ip_NotifyType

```
typedef void(* Port_Ci_Icu_Ip_NotifyType) (void)
```

The notification functions shall have no parameters and no return value.

Definition at line 133 of file Port_Ci_Icu_Ip_Types.h.

6.4.3.2 Port_Ci_Icu_Ip_CallbackType

```
typedef void(* Port_Ci_Icu_Ip_CallbackType) (uint16 callbackParam1, boolean callbackParam2)
```

Callback signature used in each channel with an active interrupt.

Definition at line 135 of file Port_Ci_Icu_Ip_Types.h.

6.4.4 Enum Reference

6.4.4.1 Port_Ci_Icu_Ip_EdgeType

enum `Port_Ci_Icu_Ip_EdgeType`

`Port_Ci_Icu_Ip_EdgeType`.

This indicates the activation type Port_Ci channel (Rising, Falling or Both)

Enumerator

<code>PORT_CI_ICU_FALLING_EDGE</code>	No trigger. An appropriate action shall be executed when a falling edge occurs on the Port_Ci input signal.
<code>PORT_CI_ICU_RISING_EDGE</code>	An appropriate action shall be executed when a rising edge occurs on the Port_Ci input signal.
<code>PORT_CI_ICU_BOTH_EDGES</code>	An appropriate action shall be executed when a rising edge or falling edge occurs on the Port_Ci input signal.

Definition at line 107 of file `Port_Ci_Icu_Ip_Types.h`.

6.4.4.2 Port_Ci_Icu_Ip_StatusType

enum `Port_Ci_Icu_Ip_StatusType`

Generic error codes.

Enumerator

<code>PORT_CI_IP_STATUS_SUCCESS</code>	Generic operation success status.
<code>PORT_CI_IP_STATUS_ERROR</code>	Generic operation failure status.

Definition at line 122 of file `Port_Ci_Icu_Ip_Types.h`.

6.4.5 Function Reference

6.4.5.1 Port_Ci_Icu_Ip_EnableNotification()

```
void Port_Ci_Icu_Ip_EnableNotification (
    uint8 instance,
    uint8 hwChannel )
```

Driver function Enable Notification for timestamp.

6.4.5.2 Port_Ci_Icu_Ip_DisableNotification()

```
void Port_Ci_Icu_Ip_DisableNotification (
    uint8 instance,
    uint8 hwChannel )
```

Driver function Disable Notification for timestamp.

6.5 CMP IPL

6.5.1 Detailed Description

CMP HW module.

The low power comparator (LPCMP) module provides a circuit for comparing two analog input voltages. It comprises a comparator (CMP), a DAC and an analog mux (ANMUX). The CMP circuit is designed to operate across the full range of the supply voltage, known as rail-to-rail operation. The DAC is a 256-tap resistor ladder network that provides a selectable voltage reference for applications requiring a voltage reference. The 256-tap resistor ladder network divides the supply reference V_{in} into 256 voltage levels. A 8-bit digital signal input selects the output voltage level, which varies from V_{in} to $V_{in}/256$. V_{in} can be selected from two voltage sources, $vrefh0$ and $vrefh1$. See the chip-specific LPCMP information for the source of $vrefh0$ and $vrefh1$.

The Analog MUX (ANMUX) provides a circuit for selecting an analog input signal from eight channels. One channel is provided by the DAC. Refer to the chip-specific LPCMP information section for details on which device resources are connected to other channels. The mux circuit is designed to operate across the full range of the supply voltage.

Data Structures

- struct [Cmp_Ip_StateType](#)
CMP IP state structure. [More...](#)

Enum Reference

- enum [Cmp_Ip_StateEType](#)
CMP initialization status.

6.5.2 Data Structure Documentation

6.5.2.1 struct Cmp_Ip_StateType

CMP IP state structure.

This structure is used by the IPL driver for internal logic. The content is populated at initialization time.

Definition at line 198 of file `Cmp_Ip_Types.h`.

6.5.3 Enum Reference

6.5.3.1 Cmp_Ip_StateEType

enum [Cmp_Ip_StateEType](#)

CMP initialization status.

Definition at line 180 of file `Cmp_Ip_Types.h`.

6.6 Icu Driver

6.6.1 Detailed Description

Data Structures

- struct [Icu_ChannelConfigType](#)
Structure that contains ICU channel configuration. [More...](#)
- struct [Icu_ConfigType](#)
This type contains initialization data. [More...](#)
- struct [Icu_DutyCycleType](#)
Structure that contains ICU Duty cycle parameters. It contains the values needed for calculating duty cycles i.e Period time value and active time value. [More...](#)

Macros

- `#define ICU\_STOPTIMESTAMP\_ID`
API service ID for [Icu_StopTimestamp](#) function.
- `#define ICU\_E\_NOT\_STARTED`
API service [Icu_StopTimestamp](#) called on a channel which was not started or already stopped.

Types Reference

- typedef uint8 [Icu_ChannelStateType](#)
ICU Channel state type.
- typedef uint16 [Icu_ChannelType](#)
This gives the numeric ID (hardware channel number) of an ICU channel.
- typedef Icu_TimerRegisterWidthType [Icu_ValueType](#)
Type for saving the timer register width value.
- typedef Icu_HwSpecificIndexType [Icu_IndexType](#)
Type for saving the ICU Hardware specific index.
- typedef Icu_HwSpecificEdgeNumberType [Icu_EdgeNumberType](#)
Type for saving hardware specific edge number.
- typedef uint32 [Icu_WakeupValueType](#)
Type for saving the Wakeup value.
- typedef uint16 [Icu_MeasurementSubModeType](#)
Type for saving the ICU measurement submode type.
- typedef void(* [Icu_NotifyType](#)) (void)
The notification functions shall have no parameters and no return value.

Enum Reference

- enum [Icu_ModeType](#)
Allow enabling or disabling of all interrupts which are not required for the ECU wakeup.
- enum [Icu_InputStateType](#)
Input state of an ICU channel.

- enum [Icu_MeasurementModeType](#)
Definition of the measurement mode type.
- enum [Icu_SignalMeasurementPropertyType](#)
Definition of the measurement property type.
- enum [Icu_TimestampBufferType](#)
Definition of the Time-stamp measurement property type.
- enum [Icu_ActivationType](#)
Definition of the type of activation of an ICU channel.
- enum [Icu_LevelType](#)
Return the status of the pin.
- enum [Icu_SelectPrescalerType](#)
Definition of prescaler type.

Function Reference

- void [Icu_Init](#) (const [Icu_ConfigType](#) *ConfigPtr)
This function initializes the driver.
- void [Icu_DeInit](#) (void)
This function de-initializes the ICU module.
- void [Icu_SetMode](#) ([Icu_ModeType](#) Mode)
This function sets the ICU mode.
- void [Icu_DisableWakeup](#) ([Icu_ChannelType](#) Channel)
This function disables the wakeup capability of a single ICU channel.
- void [Icu_EnableWakeup](#) ([Icu_ChannelType](#) Channel)
This function (re-)enables the wakeup capability of the given ICU channel.
- void [Icu_CheckWakeup](#) ([EcuM_WakeupSourceType](#) WakeupSource)
Checks if a wakeup capable ICU channel is the source for a wakeup event.
- void [Icu_SetActivationCondition](#) ([Icu_ChannelType](#) Channel, [Icu_ActivationType](#) Activation)
This function sets the activation-edge for the given channel.
- void [Icu_DisableNotification](#) ([Icu_ChannelType](#) Channel)
This function disables the notification of a channel.
- void [Icu_EnableNotification](#) ([Icu_ChannelType](#) Channel)
This function enables the notification on the given channel.
- [Icu_InputStateType](#) [Icu_GetInputState](#) ([Icu_ChannelType](#) Channel)
This function returns the status of the ICU input.
- void [Icu_StartTimestamp](#) ([Icu_ChannelType](#) Channel, [Icu_ValueType](#) *BufferPtr, uint16 BufferSize, uint16 NotifyInterval)
This function starts the capturing of timer values on the edges.
- void [Icu_StopTimestamp](#) ([Icu_ChannelType](#) Channel)
This function stops the timestamp measurement of the given channel.
- [Icu_IndexType](#) [Icu_GetTimestampIndex](#) ([Icu_ChannelType](#) Channel)
This function reads the timestamp index of the given channel.

- void [Icu_ResetEdgeCount](#) ([Icu_ChannelType](#) Channel)
This function resets the value of the counted edges to zero.
- void [Icu_EnableEdgeCount](#) ([Icu_ChannelType](#) Channel)
This function enables the counting of edges of the given channel.
- void [Icu_DisableEdgeCount](#) ([Icu_ChannelType](#) Channel)
This function disables the counting of edges of the given channel.
- [Icu_EdgeNumberType](#) [Icu_GetEdgeNumbers](#) ([Icu_ChannelType](#) Channel)
This function reads the number of counted edges.
- void [Icu_EnableEdgeDetection](#) ([Icu_ChannelType](#) Channel)
This function enables or re-enables the detection of edges of the given channel.
- void [Icu_DisableEdgeDetection](#) ([Icu_ChannelType](#) Channel)
This function disables the detection of edges of the given channel.
- [Icu_ValueType](#) [Icu_GetTimeElapsed](#) ([Icu_ChannelType](#) Channel)
This function reads the elapsed Signal Low, High or Period Time for the given channel.
- void [Icu_GetDutyCycleValues](#) ([Icu_ChannelType](#) Channel, [Icu_DutyCycleType](#) *DutyCycleValues)
This function reads the coherent active time and period time for the given ICU Channel.
- void [Icu_StartSignalMeasurement](#) ([Icu_ChannelType](#) Channel)
This function starts the measurement of signals.
- void [Icu_StopSignalMeasurement](#) ([Icu_ChannelType](#) Channel)
This function stops the measurement of signals of the given channel.
- void [Icu_GetVersionInfo](#) ([Std_VersionInfoType](#) *versioninfo)
This service returns the version information of this module.
- void [Icu_SetClockMode](#) ([Icu_SelectPrescalerType](#) selectPrescaler)
This function sets all channels prescalers based on the input mode.
- [Icu_LevelType](#) [Icu_GetInputLevel](#) ([Icu_ChannelType](#) Channel)
This function returns the actual status of PIN.
- [Icu_ValueType](#) [Icu_GetCaptureRegisterValue](#) ([Icu_ChannelType](#) Channel)
This function starts the measurement of signals.

Variables

- const [Icu_ConfigType](#) * [Icu_pCfgPtr](#) [(1U)]
Pointer initialized during init with the address of the received configuration structure.
- [Icu_ModeType](#) [Icu_CurrentMode](#) [(1U)]
Saves the current Icu mode.
- volatile [Icu_ChannelStateType](#) [Icu_aChannelState](#) [(([Icu_ChannelType](#)) 22U)]
Stores actual state and configuration of ICU Channels.

6.6.2 Data Structure Documentation

6.6.2.1 struct Icu_ChannelConfigType

Structure that contains ICU channel configuration.

It contains the information like Icu Channel Mode, Channel Notification function, overflow Notification function.

Definition at line 511 of file Icu.h.

Data Fields

- boolean [Icu_WakeupCapabile](#)
Channel wakeup capability enable.
- [Icu_ActivationType](#) [Icu_ActivEdge](#)
RISING_EDGE, FALLING_EDGE or BOTH_EDGES for EDGE_COUNTER.
- [Icu_MeasurementModeType](#) [Icu_ChannelMode](#)
EDGE_DETECT, TIME_STAMP, SIGNAL_MEASUREMENT or EDGE_COUNTER.
- [Icu_MeasurementSubModeType](#) [Icu_ChannelProperty](#)
CIRCULAR_BUFFER or LINEAR_BUFFER for TIME_STAMP, DUTY_CYCLE, HIGH_TIME, LOW_TIME or PERIOD_TIME for SIGNAL_MEASUREMENT and RISING_EDGE, FALLING_EDGE or BOTH_EDGES for EDGE_COUNTER.
- [Icu_NotifyType](#) [Icu_ChannelNotification](#)
Icu Channel Notification function for TIME_STAMP or EDGE_COUNTER mode.
- [Icu_NotifyType](#) [Icu_ChOverflowNotification](#)
Icu Channel Overflow Notification function.
- [Icu_WakeupValueType](#) [Icu_Channel_WakeupValue](#)
EcuM wakeup source Id.
- const [Icu_Ipw_ChannelConfigType](#) * [Icu_IpwChannelConfigPtr](#)
Pointer to the ipw channel pointer configuration.

Field Documentation

[Icu_WakeupCapabile](#)

boolean [Icu_WakeupCapabile](#)

Channel wakeup capability enable.

Definition at line 514 of file Icu.h.

[Icu_ActivEdge](#)

[Icu_ActivationType](#) [Icu_ActivEdge](#)

RISING_EDGE, FALLING_EDGE or BOTH_EDGES for EDGE_COUNTER.

Definition at line 516 of file Icu.h.

Icu_ChannelMode

`Icu_MeasurementModeType` `Icu_ChannelMode`

EDGE_DETECT, TIME_STAMP, SIGNAL_MEASUREMENT or EDGE_COUNTER.

Definition at line 518 of file Icu.h.

Icu_ChannelProperty

`Icu_MeasurementSubModeType` `Icu_ChannelProperty`

CIRCULAR_BUFFER or LINEAR_BUFFER for TIME_STAMP, DUTY_CYCLE, HIGH_TIME, LOW_TIME or PERIOD_TIME for SIGNAL_MEASUREMENT and RISING_EDGE, FALLING_EDGE or BOTH_EDGES for EDGE_COUNTER.

Definition at line 522 of file Icu.h.

Icu_ChannelNotification

`Icu_NotifyType` `Icu_ChannelNotification`

Icu Channel Notification function for TIME_STAMP or EDGE_COUNTER mode.

Definition at line 524 of file Icu.h.

Icu_ChOverflowNotification

`Icu_NotifyType` `Icu_ChOverflowNotification`

Icu Channel Overflow Notification function.

Definition at line 531 of file Icu.h.

Icu_Channel_WakeupValue

`Icu_WakeupValueType` `Icu_Channel_WakeupValue`

EcuM wakeup source Id.

Definition at line 534 of file Icu.h.

Icu_IpwChannelConfigPtr

`const Icu_Ipw_ChannelConfigType*` `Icu_IpwChannelConfigPtr`

Pointer to the ipw channel pointer configuration.

Definition at line 537 of file Icu.h.

6.6.2.2 struct Icu_ConfigType

This type contains initialization data.

The notification functions shall be configurable as function pointers within the initialization data structure ([Icu_ConfigType](#)). This type of the external data structure shall contain the initialization data for the ICU driver. It shall contain:

- Wakeup Module Info (in case the wakeup-capability is true)
- ICU dependent properties for used HW units
- Clock source with optional prescaler (if provided by HW)

Definition at line 551 of file Icu.h.

Data Fields

- uint8 [nNumChannels](#)
The number of configured logical channels.
- const [Icu_ChannelConfigType](#)(* [Icu_ChannelConfigPtr](#))[]
Pointer to the list of Icu configured channels.
- uint8 [nNumInstances](#)
The number of IP instances configured.
- const [Icu_Ipw_IpConfigType](#)(* [Icu_IpConfigPtr](#))[]
Pointer to the list of Icu configured channels.
- const uint8(* [Icu_IndexChannelMap](#))[]
channel index in each partition map table
- uint32 [u32UserID](#)
partition index

Field Documentation

nNumChannels

```
uint8 nNumChannels
```

The number of configured logical channels.

Definition at line 554 of file Icu.h.

Icu_ChannelConfigPtr

```
const Icu\_ChannelConfigType(* Icu\_ChannelConfigPtr ) [ ]
```

Pointer to the list of Icu configured channels.

Definition at line 557 of file Icu.h.

nNumInstances

```
uint8 nNumInstances
```

The number of IP instances configured.

Definition at line 560 of file Icu.h.

Icu_IpConfigPtr

```
const Icu_Ipw_IpConfigType(* Icu_IpConfigPtr)[]
```

Pointer to the list of Icu configured channels.

Definition at line 563 of file Icu.h.

Icu_IndexChannelMap

```
const uint8(* Icu_IndexChannelMap)[]
```

channel index in each partition map table

Definition at line 566 of file Icu.h.

u32UserID

```
uint32 u32UserID
```

partition index

Definition at line 569 of file Icu.h.

6.6.2.3 struct Icu_DutyCycleType

Structure that contains ICU Duty cycle parameters. It contains the values needed for calculating duty cycles i.e Period time value and active time value.

Definition at line 270 of file Icu_Types.h.

Data Fields

- [Icu_ValueType ActiveTime](#)

Low or High time value.

- [Icu_ValueType PeriodTime](#)

Period time value.

Field Documentation

ActiveTime

`Icu_ValueType` ActiveTime

Low or High time value.

Definition at line 272 of file Icu_Types.h.

PeriodTime

`Icu_ValueType` PeriodTime

Period time value.

Definition at line 273 of file Icu_Types.h.

6.6.3 Macro Definition Documentation

6.6.3.1 ICU_STOPTIMESTAMP_ID

```
#define ICU_STOPTIMESTAMP_ID
```

API service ID for Icu_StopTimestamp function.

Parameters used when raising an error/exception

Definition at line 482 of file Icu.h.

6.6.3.2 ICU_E_NOT_STARTED

```
#define ICU_E_NOT_STARTED
```

API service Icu_StopTimestamp called on a channel which was not started or already stopped.

Definition at line 489 of file Icu.h.

6.6.4 Types Reference

6.6.4.1 Icu_ChannelStateType

```
typedef uint8 Icu_ChannelStateType
```

ICU Channel state type.

Definition at line 219 of file Icu_Types.h.

6.6.4.2 Icu_ChannelType

```
typedef uint16 Icu_ChannelType
```

This gives the numeric ID (hardware channel number) of an ICU channel.

Definition at line 224 of file Icu_Types.h.

6.6.4.3 Icu_ValueType

```
typedef Icu_TimerRegisterWidthType Icu_ValueType
```

Type for saving the timer register width value.

Definition at line 229 of file Icu_Types.h.

6.6.4.4 Icu_IndexType

```
typedef Icu_HwSpecificIndexType Icu_IndexType
```

Type for saving the ICU Hardware specific index.

Definition at line 236 of file Icu_Types.h.

6.6.4.5 Icu_EdgeNumberType

```
typedef Icu_HwSpecificEdgeNumberType Icu_EdgeNumberType
```

Type for saving hardware specific edge number.

Definition at line 243 of file Icu_Types.h.

6.6.4.6 Icu_WakeupValueType

```
typedef uint32 Icu_WakeupValueType
```

Type for saving the Wakeup value.

Definition at line 250 of file Icu_Types.h.

6.6.4.7 Icu_MeasurementSubModeType

```
typedef uint16 Icu_MeasurementSubModeType
```

Type for saving the ICU measurement submode type.

Definition at line 257 of file Icu_Types.h.

6.6.4.8 Icu_NotifyType

```
typedef void(* Icu_NotifyType) (void)
```

The notification functions shall have no parameters and no return value.

Definition at line 262 of file Icu_Types.h.

6.6.5 Enum Reference

6.6.5.1 Icu_ModeType

enum [Icu_ModeType](#)

Allow enabling or disabling of all interrupts which are not required for the ECU wakeup.

Enumerator

ICU_MODE_NORMAL	Normal operation, all used interrupts are enabled according to the notification requests.
ICU_MODE_SLEEP	Reduced power operation. In sleep mode only those notifications are available which are configured as wakeup capable.

Definition at line 102 of file Icu_Types.h.

6.6.5.2 Icu_InputStateType

enum [Icu_InputStateType](#)

Input state of an ICU channel.

Enumerator

ICU_ACTIVE	An activation edge has been detected.
ICU_IDLE	No activation edge has been detected since the last call of Icu_GetInputState() or Icu_Init() .

Definition at line 115 of file Icu_Types.h.

6.6.5.3 Icu_MeasurementModeType

enum [Icu_MeasurementModeType](#)

Definition of the measurement mode type.

Enumerator

ICU_MODE_SIGNAL_EDGE_DETECT	Mode for detecting edges.
ICU_MODE_SIGNAL_MEASUREMENT	Mode for measuring different times between various configurable edges.
ICU_MODE_TIMESTAMP	Mode for capturing timer values on configurable edges.
ICU_MODE_EDGE_COUNTER	Mode for counting edges on configurable edges.

Definition at line 127 of file Icu_Types.h.

6.6.5.4 Icu_SignalMeasurementPropertyType

enum [Icu_SignalMeasurementPropertyType](#)

Definition of the measurement property type.

Enumerator

ICU_LOW_TIME	The channel is configured for reading the elapsed Signal Low Time.
ICU_HIGH_TIME	The channel is configured for reading the elapsed Signal High Time.
ICU_PERIOD_TIME	The channel is configured for reading the elapsed Signal Period Time.
ICU_DUTY_CYCLE	The channel is configured to read values which are needed for calculating the duty cycle (coherent Active and Period Time).

Definition at line 143 of file Icu_Types.h.

6.6.5.5 Icu_TimestampBufferType

enum `Icu_TimestampBufferType`

Definition of the Time-stamp measurement property type.

Enumerator

ICU_LINEAR_BUFFER	The buffer will just be filled once.
ICU_CIRCULAR_BUFFER	After reaching the end of the buffer, the driver restarts at the beginning of the buffer.

Definition at line 162 of file Icu_Types.h.

6.6.5.6 Icu_ActivationType

enum `Icu_ActivationType`

Definition of the type of activation of an ICU channel.

Enumerator

ICU_RISING_EDGE	An appropriate action shall be executed when a rising edge occurs on the ICU input signal.
ICU_FALLING_EDGE	An appropriate action shall be executed when a falling edge occurs on the ICU input signal.
ICU_BOTH_EDGES	An appropriate action shall be executed when either a rising or falling edge occur on the ICU input signal.

Definition at line 173 of file Icu_Types.h.

6.6.5.7 Icu_LevelType

enum `Icu_LevelType`

Return the status of the pin.

Enumeration of to check the status of pin.

Enumerator

ICU_LEVEL_LOW	Default Input PIN Status.
ICU_LEVEL_HIGH	As Icu_GetInputState do not give the Actual PIN status user can call the Non Autosar API Icu_GetInputLevel to get the Actual status of PIN.

Definition at line 189 of file Icu_Types.h.

6.6.5.8 Icu_SelectPrescalerType

```
enum Icu_SelectPrescalerType
```

Definition of prescaler type.

Enumerator

ICU_NORMAL_CLOCK_MODE	Default channel prescaler.
ICU_ALTERNATE_CLOCK_MODE	Alternate channel prescaler mode.

Definition at line 206 of file Icu_Types.h.

6.6.6 Function Reference

6.6.6.1 Icu_Init()

```
void Icu_Init (
    const Icu_ConfigType * ConfigPtr )
```

This function initializes the driver.

This service is a non reentrant function used for driver initialization. The Initialization function shall initialize all relevant registers of the configured hardware with the values of the structure referenced by the parameter ConfigPtr. If the hardware allows for only one usage of the register, the driver module implementing that functionality is responsible for initializing the register. The initialization function of this module shall always have a pointer as a parameter, even though for Variant PC no configuration set shall be given. Instead a NULL pointer shall be passed to the initialization function. The Icu module environment shall not call Icu_Init during a running operation (e. g. timestamp measurement or edge counting).

Parameters

in	ConfigPtr	Pointer to a selected configuration structure.
----	-----------	--

Returns

void

6.6.6.2 Icu_DeInit()

```
void Icu_DeInit (
```

```
void )
```

This function de-initializes the ICU module.

This service is a Non reentrant function used for ICU De-Initialization After the call of this service, the state of the peripherals used by configuration shall be the same as after power on reset. Values of registers which are not writable are excluded. This service shall disable all used interrupts and notifications. The Icu module environment shall not call Icu_DeInit during a running operation (e. g. timestamp measurement or edge counting)

Returns

```
void
```

Precondition

Icu_Init must be called before.

6.6.6.3 Icu_SetMode()

```
void Icu_SetMode (
    Icu_ModeType Mode )
```

This function sets the ICU mode.

This service is a non reentrant function used for ICU mode selection. This service shall set the operation mode to the given mode parameter. This service can be called during running operations. If so, an ongoing operation that generates interrupts on a wakeup capable channel like e.g. time stamping or edge counting might lead to the ICU module not being able to properly enter sleep mode. This is then a system or ECU configuration issue not a problem of this specification.

Parameters

in	<i>Mode</i>	Specifies the operation mode
----	-------------	------------------------------

Returns

```
void
```

Precondition

Icu_Init must be called before.

6.6.6.4 Icu_DisableWakeup()

```
void Icu_DisableWakeup (
    Icu_ChannelType Channel )
```

This function disables the wakeup capability of a single ICU channel.

This service is reentrant function and shall disable the wakeup capability of a single ICU channel. This service is only feasible for ICU channels configured statically as wakeup capable true. The function Icu_DisableWakeup shall be pre compile time configurable On,Off by the configuration parameter IcuDisableWakeupApi.

Parameters

in	<i>Channel</i>	Logical number of the ICU channel
----	----------------	-----------------------------------

Returns

void

Precondition

Icu_Init must be called before.

6.6.6.5 Icu_EnableWakeup()

```
void Icu_EnableWakeup (
    Icu_ChannelType Channel )
```

This function (re-)enables the wakeup capability of the given ICU channel.

The function is reentrant and re-enable the wake-up capability of a single ICU channel.

Parameters

in	<i>Channel</i>	Logical number of the ICU channel
----	----------------	-----------------------------------

Returns

void

Precondition

Icu_Init must be called before. The channel must be configured as wakeup capable.

6.6.6.6 Icu_CheckWakeup()

```
void Icu_CheckWakeup (
    Ecum_WakeupSourceType WakeupSource )
```

Checks if a wakeup capable ICU channel is the source for a wakeup event.

The function calls the ECU state manager service Ecum_SetWakeupEvent in case of a valid ICU channel wakeup event.

Parameters

in	<i>WakeupSource</i>	Information on wakeup source to be checked.
----	---------------------	---

Returns

void

Precondition

Icu_Init must be called before. The channel must be configured as wakeup capable.

6.6.6.7 Icu_SetActivationCondition()

```
void Icu_SetActivationCondition (
    Icu_ChannelType Channel,
    Icu_ActivationType Activation )
```

This function sets the activation-edge for the given channel.

This service is reentrant and shall set the activation-edge according to Activation parameter for the given channel. This service shall support channels which are configured for the following Icu_MeasurementMode:

- ICU_MODE_SIGNAL_EDGE_DETECT
- ICU_MODE_TIMESTAMP
- ICU_MODE_EDGE_COUNTER

Parameters

in	<i>Channel</i>	Logical number of the ICU channel
in	<i>Activation</i>	Type of activation.

Returns

void

Precondition

Icu_Init must be called before. The channel must be properly configured (ICU_MODE_SIGNAL_EDGE_DETECT, ICU_MODE_TIMESTAMP, ICU_MODE_EDGE_COUNTER).

6.6.6.8 Icu_DisableNotification()

```
void Icu_DisableNotification (
    Icu_ChannelType Channel )
```

This function disables the notification of a channel.

This function is reentrant and disables the notification of a channel.

Parameters

in	<i>Channel</i>	Logical number of the ICU channel
----	----------------	-----------------------------------

Returns

void

Precondition

Icu_Init must be called before.

6.6.6.9 Icu_EnableNotification()

```
void Icu_EnableNotification (
    Icu_ChannelType Channel )
```

This function enables the notification on the given channel.

This function is reentrant and enables the notification on the given channel. The notification will be reported only when the channel measurement property is enabled or started

Parameters

in	<i>Channel</i>	Logical number of the ICU channel
----	----------------	-----------------------------------

Returns

void

Precondition

Icu_Init must be called before.

6.6.6.10 Icu_GetInputState()

```
Icu_InputStateType Icu_GetInputState (
    Icu_ChannelType Channel )
```

This function returns the status of the ICU input.

This service is reentrant shall return the status of the ICU input. Only channels which are configured for the following Icu_MeasurementMode shall be supported:

- ICU_MODE_SIGNAL_EDGE_DETECT,
- ICU_MODE_SIGNAL_MEASUREMENT.

Parameters

in	Channel	Logical number of the ICU channel
----	---------	-----------------------------------

Returns

Icu_InputStateType

Return values

ICU_ACTIVE	An activation edge has been detected
ICU_IDLE	No activation edge has been detected since the last call of Icu_GetInputState() or Icu_Init().

Precondition

Icu_Init must be called before.

6.6.6.11 Icu_StartTimestamp()

```
void Icu_StartTimestamp (
    Icu_ChannelType Channel,
    Icu_ValueType * BufferPtr,
    uint16 BufferSize,
    uint16 NotifyInterval )
```

This function starts the capturing of timer values on the edges.

This function is reentrant and starts the capturing of timer values on the edges activated by the service Icu_SetActivationCondition() to an external buffer.

Parameters

in	Channel	Logical number of the ICU channel
in	BufferPtr	Pointer to the buffer-array where the timestamp values shall be placed.

Parameters

in	<i>BufferSize</i>	Size of the external buffer (number of entries)
in	<i>NotifyInterval</i>	Notification interval (number of events).

Returns

void

Precondition

Icu_Init must be called before.

6.6.6.12 Icu_StopTimestamp()

```
void Icu_StopTimestamp (
    Icu_ChannelType Channel )
```

This function stops the timestamp measurement of the given channel.

This function is reentrant and stops the timestamp measurement of the given channel.

Parameters

in	<i>Channel</i>	Logical number of the ICU channel
----	----------------	-----------------------------------

Returns

void

Precondition

Icu_Init must be called before.

6.6.6.13 Icu_GetTimestampIndex()

```
Icu_IndexType Icu_GetTimestampIndex (
    Icu_ChannelType Channel )
```

This function reads the timestamp index of the given channel.

This service is reentrant and read the timestamp index of the given channel, which is the next to be written. This function shall return "0" in case the service is called before [Icu_StartTimestamp\(\)](#)

Parameters

in	<i>Channel</i>	Logical number of the ICU channel
----	----------------	-----------------------------------

Returns

Icu_IndexType - Abstract return type to cover different microcontrollers.

Precondition

Icu_Init must be called before the given channel must be configured in Measurement Mode ICU_MODE_TIMESTAMP. Icu_StartTimestamp must be called before.

6.6.6.14 Icu_ResetEdgeCount()

```
void Icu_ResetEdgeCount (
    Icu_ChannelType Channel )
```

This function resets the value of the counted edges to zero.

This function is reentrant and resets the value of the counted edges to zero.

Parameters

in	<i>Channel</i>	Logical number of the ICU channel.
----	----------------	------------------------------------

Returns

void

Precondition

Icu_Init must be called before.

6.6.6.15 Icu_EnableEdgeCount()

```
void Icu_EnableEdgeCount (
    Icu_ChannelType Channel )
```

This function enables the counting of edges of the given channel.

This service is reentrant and shall enable the counting of edges of the given channel. Note: This service does not do the real counting itself. This is done by the hardware (capture unit). Only the configured edges shall be counted (rising edge, falling edge or both edges).

Configuration of the edge is done in Icu_Init or Icu_SetActivationCondition. The configured edge can be changed during runtime using Icu_SetActivationCondition. Interrupts are not required for edge counting. If interrupts are enabled, the interrupt service routine will set the overflow flag if more than 0xFFFF edges are measured.

Parameters

in	<i>Channel</i>	Logical number of the ICU channel
----	----------------	-----------------------------------

Returns

void

Precondition

Icu_Init must be called before. The given channel must be configured in Measurement Mode Edge Counter.

6.6.6.16 Icu_DisableEdgeCount()

```
void Icu_DisableEdgeCount (
```

```
Icu_ChannelType Channel )
```

This function disables the counting of edges of the given channel.

This function is reentrant and disables the counting of edges of the given channel.

Parameters

in	<i>Channel</i>	Logical number of the ICU channel
----	----------------	-----------------------------------

Returns

void

Precondition

Icu_Init must be called before. The given channel must be configured in Measurement Mode Edge Counter.

6.6.6.17 Icu_GetEdgeNumbers()

```
Icu_EdgeNumberType Icu_GetEdgeNumbers (
    Icu_ChannelType Channel )
```

This function reads the number of counted edges.

This function is reentrant reads the number of counted edges after the last call of [Icu_ResetEdgeCount\(\)](#).

Parameters

in	<i>Channel</i>	Logical number of the ICU channel
----	----------------	-----------------------------------

Returns

Icu_EdgeNumberType - Number of the counted edges.

Precondition

Icu_Init must be called before. The given channel must be configured in Measurement Mode Edge Counter.

6.6.6.18 Icu_EnableEdgeDetection()

```
void Icu_EnableEdgeDetection (
    Icu_ChannelType Channel )
```

This function enables or re-enables the detection of edges of the given channel.

This function is reentrant enables or re-enables the detection of edges of the given channel.

Parameters

in	<i>Channel</i>	Logical number of the ICU channel
----	----------------	-----------------------------------

Returns

void

Precondition

Icu_Init must be called before. The channel must be configured in Measurement Mode Edge Counter

6.6.6.19 Icu_DisableEdgeDetection()

```
void Icu_DisableEdgeDetection (
    Icu_ChannelType Channel )
```

This function disables the detection of edges of the given channel.

This function is reentrant and disables the detection of edges of the given channel.

Parameters

in	<i>Channel</i>	Logical number of the ICU channel
----	----------------	-----------------------------------

Returns

void

Precondition

Icu_Init must be called before. The channel must be configured in Measurement Mode Edge Detection.

6.6.6.20 Icu_GetTimeElapsed()

```
Icu_ValueType Icu_GetTimeElapsed (
    Icu_ChannelType Channel )
```

This function reads the elapsed Signal Low, High or Period Time for the given channel.

This service is reentrant and reads the elapsed Signal Low Time for the given channel that is configured in Measurement Mode Signal Measurement, Signal Low Time. The elapsed time is measured between a falling edge and the consecutive rising edge of the channel. This service reads the elapsed Signal High Time for the given channel that is configured in Measurement Mode Signal Measurement, Signal High Time. The elapsed time is measured between a rising edge and the consecutive falling edge of the channel. This service reads the elapsed Signal Period Time for the given channel that is configured in Measurement Mode Signal Measurement, Signal Period Time. The elapsed time is measured between consecutive rising (or falling) edges of the channel. The period start edge is

configurable.

Parameters

in	<i>Channel</i>	Logical number of the ICU channel
----	----------------	-----------------------------------

Returns

Icu_ValueType - the elapsed Signal Low Time for the given channel that is configured in Measurement Mode Signal Measurement, Signal Low Time

Precondition

Icu_Init must be called before. The channel must be configured in Measurement Mode Signal Measurement.

6.6.6.21 Icu_GetDutyCycleValues()

```
void Icu_GetDutyCycleValues (
    Icu_ChannelType Channel,
    Icu_DutyCycleType * DutyCycleValues )
```

This function reads the coherent active time and period time for the given ICU Channel.

The function is reentrant and reads the coherent active time and period time for the given ICU Channel, if it is configured in Measurement Mode Signal Measurement, Duty Cycle Values.

Parameters

in	<i>Channel</i>	Logical number of the ICU channel
out	<i>DutyCycleValues</i>	Pointer to a buffer where the results (high time and period time) shall be placed.

Returns

void

Precondition

Icu_Init must be called before. The given channel must be configured in Measurement Mode Signal Measurement, Duty Cycle Values.

6.6.6.22 Icu_StartSignalMeasurement()

```
void Icu_StartSignalMeasurement (
    Icu_ChannelType Channel )
```

This function starts the measurement of signals.

This service is reentrant and starts the measurement of signals beginning with the configured default start edge which occurs first after the call of this service. This service shall only be available in Measurement Mode ICU_M←ODE_SIGNAL_MEASUREMENT. This service shall reset the state for the given channel to ICU_IDLE.

Parameters

in	<i>Channel</i>	Logical number of the ICU channel
----	----------------	-----------------------------------

Returns

void

Precondition

Icu_Init must be called before. The channel must be configured in Measurement Mode Signal Measurement.

6.6.6.23 Icu_StopSignalMeasurement()

```
void Icu_StopSignalMeasurement (
    Icu_ChannelType Channel )
```

This function stops the measurement of signals of the given channel.

This function is reentrant and stops the measurement of signals of the given channel.

Parameters

in	<i>Channel</i>	- Logical number of the ICU channel
----	----------------	-------------------------------------

Returns

void

Precondition

Icu_Init must be called before. The channel must be configured in Measurement Mode Signal Measurement.

6.6.6.24 Icu_GetVersionInfo()

```
void Icu_GetVersionInfo (
    Std_VersionInfoType * versioninfo )
```

This service returns the version information of this module.

This service is Non reentrant and returns the version information of this module. The version information includes:

- Module Id
- Vendor Id
- Vendor specific version numbers If source code for caller and callee of this function is available this function should be realized as a macro. The macro should be defined in the modules header file.

Parameters

out	<i>versioninfo</i>	Pointer to location to store version info
-----	--------------------	---

Returns

void

6.6.6.25 Icu_SetClockMode()

```
void Icu_SetClockMode (
    Icu_SelectPrescalerType selectPrescaler )
```

This function sets all channels prescalers based on the input mode.

Parameters

<i>selectPrescaler</i>	Select the used prescaler: prescaler/alternatePrescaler.
------------------------	--

Returns

void

Precondition

Icu_Init must be called before.

6.6.6.26 Icu_GetInputLevel()

```
Icu_LevelType Icu_GetInputLevel (
    Icu_ChannelType Channel )
```

This function returns the actual status of PIN.

This function returns the actual status of PIN.

Parameters

in	<i>Channel</i>	Logical number of the ICU channel
----	----------------	-----------------------------------

Returns

Icu_LevelType

Precondition

Icu_Init must be called before.

6.6.6.27 Icu_GetCaptureRegisterValue()

```
Icu_ValueType Icu_GetCaptureRegisterValue (
    Icu_ChannelType Channel )
```

This function starts the measurement of signals.

. This service returns the value of Capture register. This API is used to measure the time difference between 2 different timer channels.

Parameters

in	<i>Channel</i>	Logical number of the ICU channel
----	----------------	-----------------------------------

Returns

Icu_ValueType Value of Capture register

Precondition

Icu_Init must be called before.

The given channel must be configured in SignalMeasurement or in Timestamp mode

6.6.7 Variable Documentation

6.6.7.1 Icu_pCfgPtr

```
const Icu_ConfigType* Icu_pCfgPtr[(1U)] [extern]
```

Pointer initialized during init with the address of the received configuration structure.

Will be used by all functions to access the configuration data.

6.6.7.2 Icu_CurrentMode

```
Icu_ModeType Icu_CurrentMode[(1U)] [extern]
```

Saves the current Icu mode.

6.6.7.3 Icu_aChannelState

```
volatile Icu_ChannelStateType Icu_aChannelState[((Icu_ChannelType) 22U)] [extern]
```

Stores actual state and configuration of ICU Channels.

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Limited warranty and liability - Information in this document is believed to be accurate and reliable. However, NXP Semiconductors does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information. NXP Semiconductors takes no responsibility for the content in this document if provided by an information source outside of NXP Semiconductors.

In no event shall NXP Semiconductors be liable for any indirect, incidental, punitive, special or consequential damages (including - without limitation - lost profits, lost savings, business interruption, costs related to the removal or replacement of any products or rework charges) whether or not such damages are based on tort (including negligence), warranty, breach of contract or any other legal theory.

Notwithstanding any damages that customer might incur for any reason whatsoever, NXP Semiconductors' aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms and conditions of commercial sale of NXP Semiconductors.

Right to make changes - NXP Semiconductors reserves the right to make changes to information published in this document, including without limitation specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

Suitability for use in automotive applications - This NXP product has been qualified for use in automotive applications. If this product is used by customer in the development of, or for incorporation into, products or services (a) used in safety critical applications or (b) in which failure could lead to death, personal injury, or severe physical or environmental damage (such products and services hereinafter referred to as "Critical Applications"), then customer makes the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, safety, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP. As such, customer assumes all risk related to use of any products in Critical Applications and NXP and its suppliers shall not be liable for any such use by customer. Accordingly, customer will indemnify and hold NXP harmless from any claims, liabilities, damages and associated costs and expenses (including attorneys' fees) that NXP may incur related to customer's incorporation of any product in a Critical Application.

Applications - Applications that are described herein for any of these products are for illustrative purposes only. NXP Semiconductors makes no representation or warranty that such applications will be suitable for the specified use without further testing or modification.

Customers are responsible for the design and operation of their applications and products using NXP Semiconductors products, and NXP Semiconductors accepts no liability for any assistance with applications or customer product design. It is customer's sole responsibility to determine whether the NXP Semiconductors product is suitable and fit for the customer's applications and products planned, as well as for the planned application and use of customer's third party customer(s). Customers should provide appropriate design and operating safeguards to minimize the risks associated with their applications and products.

NXP Semiconductors does not accept any liability related to any default, damage, costs or problem which is based on any weakness or default in the customer's applications or products, or the application or use by customer's third party customer(s). Customer is responsible for doing all necessary testing for the customer's applications and products using NXP Semiconductors products in order to avoid a default of the applications and the products or of the application or use by customer's third party customer(s). NXP does not accept any liability in this respect.

Terms and conditions of commercial sale - NXP Semiconductors products are sold subject to the general terms and conditions of commercial sale, as published at <http://www.nxp.com/profile/terms>, unless otherwise agreed in a valid written individual agreement. In case an individual agreement is concluded only the terms and conditions of the respective agreement shall apply. NXP Semiconductors hereby expressly objects to applying the customer's general terms and conditions with regard to the purchase of NXP Semiconductors products by customer.

Export control - This document as well as the item(s) described herein may be subject to export control regulations. Export might require a prior authorization from competent authorities.

Translations — A non-English (translated) version of a document, including the legal information in that document, is for reference only. The English version shall prevail in case of any discrepancy between the translated and English versions.

Security — Customer understands that all NXP products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Customer is responsible for the design and operation of its applications and products throughout their lifecycles to reduce the effect of these vulnerabilities on customer's applications and products. Customer's responsibility also extends to other open and/or proprietary technologies supported by NXP products for use in customer's applications. NXP accepts no liability for any vulnerability. Customer should regularly check security updates from NXP and follow up appropriately.

Customer shall select products with security features that best meet rules, regulations, and standards of the intended application and make the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP.

NXP has a Product Security Incident Response Team (PSIRT) (reachable at <mailto:PSIRT@nxp.com>) that manages the investigation, reporting, and solution release to security vulnerabilities of NXP products.

NXP B.V. - NXP B.V. is not an operating company and it does not distribute or sell products.